AI-Based Paddy Seed Quality Assessment And Farmer Feedback System

A thesis submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

By

Joymangol Chingangbam	**(2201CS0004)**
Nilambar Elangbam	**(2201CS0005)**
Khangembam Yaiphaba Singh	**(2201CS0102)**
Dayananda Laishram	**(2201CS0113)**

Under the Guidance of

Dr. Roshni Rajkumari
Assistant Professor

Department of Computer Science & Engineering

Manipur Technical University

May, 2026

DEDICATION

We dedicate this thesis to everyone who supported us and helped make this project possible.

To our parents and families: For your patience, encouragement, and understanding when the project demanded long hours and late nights. Your support kept us going.

To our supervisor and teachers: For sharing your knowledge, guiding our efforts, and keeping us on track when we faced technical challenges.

To our friends and classmates: For the shared ideas, coffee runs, and the motivation to keep working through every bug and deadline.

To the open-source developers and researchers in computer vision and agricultural tech: Your libraries, models, and shared datasets formed the starting point for our work and inspired us to build tools that help our local community.

This work is a reflection of the guidance, help, and encouragement we received from all of you throughout our college years.

AI-Based Paddy Seed Quality Assessment And Farmer Feedback System

A thesis submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

By

Joymangol Chingangbam	**(2201CS0004)**
Nilambar Elangbam	**(2201CS0005)**
Khangembam Yaiphaba Singh	**(2201CS0102)**
Dayananda Laishram	**(2201CS0113)**

Under the Guidance of

Dr. Roshni Rajkumari
Assistant Professor

Department of Computer Science & Engineering

Manipur Technical University

May, 2026

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

BONAFIDE CERTIFICATE

This is to certify that the thesis entitled **"AI-Based Paddy Seed Quality Assessment And Farmer Feedback System"** submitted by:

Joymangol Chingangbam	**(2201CS0004)**
Nilambar Elangbam	**(2201CS0005)**
Khangembam Yaiphaba Singh	**(2201CS0102)**
Dayananda Laishram	**(2201CS0113)**

to the **Department of Computer Science & Engineering**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** degree in **Computer Science and Engineering**, is a genuine and original work carried out by them under my supervision during the academic year **2025–2026**.

The project embodies the results of their own research and development work and has not been submitted, either in part or in full, for the award of any other degree or diploma of this university or any other institution. The work has been carried out with sincerity, academic integrity, and adherence to research ethics.

We consider this work worthy of consideration for the partial fulfilment of the requirements for the said degree.

Supervisor

Dr. Roshni Rajkumari
Assistant Professor,
Computer Science & Engineering

Head of Department

Mrs. Tayenjam Aerena
Assistant Professor & Head,
Computer Science & Engineering

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

DECLARATION

I hereby declare that the thesis entitled **"AI-Based Paddy Seed Quality Assessment And Farmer Feedback System"**, submitted to the **Department of Computer Science and Engineering, Manipur Technical University**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** degree, is a result of my own work and effort. Under the guidance of **Dr. Roshni Rajkumari**, Assistant Professor, Department of Computer Science and Engineering, Manipur Technical University.

I affirm that this work has been carried out with academic honesty and integrity. All sources of information have been properly cited and acknowledged. This thesis has not been submitted earlier, either in part or in full, for the award of any degree or diploma at this or any other institution.

Date:
Place:

Joymangol Chingangbam
Reg. No. - 2201CS0004
Computer Science & Engineering

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

DECLARATION

I hereby declare that the thesis entitled **"AI-Based Paddy Seed Quality Assessment And Farmer Feedback System"**, submitted to the **Department of Computer Science and Engineering, Manipur Technical University**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** degree, is a result of my own work and effort. Under the guidance of **Dr. Roshni Rajkumari**, Assistant Professor, Department of Computer Science and Engineering, Manipur Technical University.

I affirm that this work has been carried out with academic honesty and integrity. All sources of information have been properly cited and acknowledged. This thesis has not been submitted earlier, either in part or in full, for the award of any degree or diploma at this or any other institution.

Date:
Place:

Nilambar Elangbam
Reg. No. - 2201CS0005
Computer Science & Engineering

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

DECLARATION

I hereby declare that the thesis entitled **"AI-Based Paddy Seed Quality Assessment And Farmer Feedback System"**, submitted to the **Department of Computer Science and Engineering, Manipur Technical University**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** degree, is a result of my own work and effort. Under the guidance of **Dr. Roshni Rajkumari**, Assistant Professor, Department of Computer Science and Engineering, Manipur Technical University.

I affirm that this work has been carried out with academic honesty and integrity. All sources of information have been properly cited and acknowledged. This thesis has not been submitted earlier, either in part or in full, for the award of any degree or diploma at this or any other institution.

Date:
Place:

Khangembam Yaiphaba Singh
Reg. No. - 2201CS0102
Computer Science & Engineering

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

DECLARATION

I hereby declare that the thesis entitled **"AI-Based Paddy Seed Quality Assessment And Farmer Feedback System"**, submitted to the **Department of Computer Science and Engineering, Manipur Technical University**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** degree, is a result of my own work and effort. Under the guidance of **Dr. Roshni Rajkumari**, Assistant Professor, Department of Computer Science and Engineering, Manipur Technical University.

I affirm that this work has been carried out with academic honesty and integrity. All sources of information have been properly cited and acknowledged. This thesis has not been submitted earlier, either in part or in full, for the award of any degree or diploma at this or any other institution.

Date:
Place:

Dayananda Laishram
Reg. No. - 2201CS0113
Computer Science & Engineering

MANIPUR TECHNICAL UNIVERSITY

(A University established under the **Manipur Technical University Act, 2016**)

Imphal, Manipur – 795004

ACKNOWLEDGEMENT

We would like to express our gratitude to everyone who helped us complete our final year project and thesis on **"AI-Based Paddy Seed Quality Assessment And Farmer Feedback System".**

We are grateful to our Vice Chancellor, **Prof. W. Chandbabu Singh**, for his support and encouragement throughout our time at Manipur Technical University. We also thank the **Department of Computer Science and Engineering** for providing the laboratory facilities, computing resources, and academic environment that made our project development possible.

Our sincere thanks go to our project supervisor, **Dr. Roshni Rajkumari**, Assistant Professor, for her valuable guidance, detailed reviews, and constant encouragement throughout the design, training, and deployment of our models. We are also grateful to **Mrs. Tayenjam Aerena**, Head of Department, for her guidance and support.

Finally, we thank our classmates, the technical staff in our department, and our friends for their help and feedback during our project work.

Date:
Place:

Sincerely,
Joymangol Chingangbam
Nilambar Elangbam
Dayananda Laishram
Khangembam Yaiphaba Singh

ABSTRACT

The quality of paddy seed plays a vital role in crop yield, germination rate and overall agricultural productivity. Many small and marginal farmers suffer crop loss due to substandard or counterfeit seeds that closely resemble certified products. Due to the lack of affordable and rapid verification methods, farmers rely on packaging labels and basic manual inspection, which is time consuming, inconsistent and error prone. Poor quality seeds are generally identified only after sowing, resulting in low germination rates, reduced yield and financial losses.

To address these challenges, **AgriVerify**, an AI-powered Paddy Seed Quality Assessment and Farmer Feedback System, is designed to combat the spread of counterfeit and substandard paddy seeds. The system is a web-based platform that utilizes artificial intelligence and deep learning models to automate the detection of seed quality into healthy and suspicious categories based on characteristics such as size, shape uniformity, colour, surface texture and visible defects. Farmers can scan seed samples using a simple image-based interface to receive an estimated quality assessment presented in clear, farmer-friendly language.

In addition to seed quality assessment, the system includes a Farmer Complaint and Feedback module that enables reporting of field-level issues such as poor germination, crop failure, cultivation experiences and seed performance.

The system avoids chemical analysis, specialized hardware and complex supply chain systems, making it low-cost, scalable and suitable for rural deployment. It helps farmers select better quality seeds, reduce crop failure and improve productivity. The integration of artificial intelligence with a farmer feedback mechanism makes **AgriVerify** a practical and scalable solution for modern agricultural practices and sustainable farming development.

Table of Contents

BONAFIDE CERTIFICATE i

DECLARATION ii

ACKNOWLEDGEMENT vi

ABSTRACT vii

List of Tables xiii

List of Figures xv

Symbols and Acronyms 1

1 Introduction 1

1.1 Background and Context 1

1.2 Motivation 1

1.3 Problem Statement 2

1.4 Objectives 2

1.5 Scope of the Project 3

2 Literature Review 4

2.1 Seed Quality Assessment: Traditional Methods 4

2.2 Deep Learning for Agricultural Image Detection 4

2.3 Transfer Learning and ResNet Architectures 5

2.4 Farmer Feedback Systems 5

2.5 Web-Based Agricultural Platforms 6
2.6 Review of Related Work 6
2.7 Summary and Research Gaps 7

3 System Design and Architecture 8
3.1 Overall System Architecture and Technology Stack 8
3.2 Functional and Non-Functional Requirements 10
3.3 High-Level System Architecture 11
3.4 System Data Flow Pipeline 11
3.5 Database Entity-Relationship Model 12

4 Deep Learning Model for Paddy Seed Quality Assessment 15
4.1 Dataset Description 15
4.1.1 Data Sources and Collection (Kaggle) 15
4.1.1.1 Public Source 15
4.1.1.2 Custom Source 16
4.1.2 Dataset Statistics and Class Distribution 17
4.1.3 Train/Validation/Test Split Strategy 18
4.1.4 Data Preprocessing and Augmentation 18
4.2 Model Architecture 19
4.2.1 ResNet50 Backbone (25.6M Parameters) 19
4.2.2 Custom Model Head 20
4.2.3 Anti-Overfitting Techniques 20
4.3 DUS Parameter Integration 21
4.3.1 Feature Extraction—17 Physical Parameters 21
4.3.2 Pixel-to-Physical Unit Conversion 22
4.3.3 Variety Matching (12 Reference Varieties) 22
4.4 Training Pipeline 23
4.4.1 Phase 1—Model Head Training (Epochs 1–8) 23
4.4.2 Phase 2—Full Fine-Tuning (Epochs 9–15) 23

4.4.3 Hyperparameter Configuration and Optimiser 23
4.4.4 Model Export Formats 24
4.5 Model Comparison and Selection 24
4.6 Final Model Performance Summary 26

5 Model Serving API **28**
5.1 API Architecture and Request Orchestration 28
5.1.1 FastAPI and ASGI Serving Stack 28
5.1.2 End-to-End Model Request Flow 29
5.2 API Specification and Endpoint Contracts 30
5.3 Containerisation and Deployment Architecture 30
5.3.1 CI/CD Deployment Pipeline 31
5.3.2 Live Production API Endpoint 32

6 Backend System Architecture and Implementation **33**
6.1 Architectural Design 33
6.2 Technology Stack 34
6.3 Project Structure 35
6.4 Domain Layer 36
6.4.1 Core Entities 36
6.4.2 Interfaces 36
6.5 Application Layer 36
6.5.1 Application Services 37
6.5.2 DTOs 37
6.5.3 Validation 37
6.6 Infrastructure Layer 38
6.6.1 Database Management and Repositories 38
6.6.2 AI and Cloud Integrations 38
6.7 Database Schema 39
6.8 Presentation Layer 40

6.8.1 API Controllers 40
6.8.2 Middleware 40
6.9 Authentication and Security 41
6.10 Verification Workflow 41
6.11 Configuration and Deployment 42

7 Frontend System Architecture and Implementation 44
7.1 Frontend Architecture and Design Patterns 44
7.1.1 Architectural Layers 44
7.1.2 Secure Proxy Architecture 45
7.2 Technology Stack Analysis 45
7.3 Modular Project Structure 46
7.4 User Role-Based Access Control (RBAC) 46
7.4.1 Role Partitioning 46
7.4.2 Route Protection Middleware 46
7.5 Authentication and Secure Persistence 47
7.5.1 Token Storage Architecture 47
7.5.2 Automated Token Rotation 47
7.6 API Integration and Type Safety 48
7.7 State Management 48
7.7.1 Asynchronous Server State (TanStack Query) 48
7.7.2 Synchronous Client State (Zustand) 49
7.8 Responsive UI Design and Field Usability 49
7.8.1 Standardized UI Library 49
7.8.2 Accessibility Enhancements 49

8 System Integration 50
8.1 System Integration 50
8.2 End-to-End Architectural Workflow 50
8.2.1 Unexpected Integration Challenges and Resolutions 53

8.2.2 Live Prototype Deployment 54

9 Results and Evaluation **55**

9.1 Model Performance 55

9.1.1 Accuracy, Precision, Recall, and F1 Score 55

9.2 Seed Quality Results: Pure vs. Impure Seeds 59

9.3 Confusion Matrix Analysis 60

9.4 Regularization Effectiveness 61

9.5 Model Comparison and Architecture Selection 62

9.6 System Performance and Production Latency 64

9.6.1 Inference Latency Evaluation 64

9.6.2 End-to-End System Performance Under Load 65

9.7 Functional Verification and Test Execution 65

9.7.1 Seed Quality Assessment Validation 66

9.7.2 Grievance Management Subsystem Validation 66

9.7.3 Role-Based Access Control and Security Verification 66

9.8 System User Interfaces and Operational Workflows 67

9.8.1 Public Portal, Onboarding, and Multilingual Support 67

9.8.2 Farmer Portal and AI Seed Quality Assessment Workflow . . 70

9.8.3 Grievance Redressal and Officer Investigation Interfaces . . . 73

9.8.4 Platform Administration, User Governance and Telemetry . 76

9.9 Challenges Encountered and Mitigation Strategies 76

10 Conclusion And Future Work **79**

10.1 Summary of Work 79

10.2 Key Contributions and Findings 80

10.3 Future Enhancements 81

References **82**

List of Tables

3.1 Backend Architectural Layers and Responsibilities 8
3.2 Technology Stack of AgriVerify 9
3.3 System Functional Requirements 10
3.4 System Non-Functional Requirements 10

4.1 Dataset Source Summary 17
4.2 Class-wise Distribution of the Combined Dataset 17
4.3 Dataset Split Statistics 18
4.4 Hyperparameter Configuration and Training Settings 24
4.5 Model Export Formats 24
4.6 Architecture Comparison on Test Set 25
4.7 Final Test Set Performance—ResNet50 26
4.8 Training Behaviour Summary 27

5.1 API Specification for the `POST /predict` Model Inference Endpoint 30
5.2 API Specification for the `GET /health` Liveness Endpoint 31

6.1 Backend Technology Stack 35
6.2 Backend Project Structure 35
6.3 Important DTOs 37
6.4 External Service Integrations 38
6.5 API Controllers 40

7.1 Frontend Technology Stack 46
7.2 Standardized UI Components 49

8.1 System Integration and Communication Matrix 51

9.1 Overall Model Evaluation Results 56
9.2 Performance on Each Seed Category 59
9.3 Detailed Look at Every Prediction 60
9.4 How Our Anti-Overfitting Techniques Helped 61
9.5 Comparing Four Different Neural Network Architectures 62
9.6 Speed on Different Hardware 65
9.7 Real-World System Performance Under Load 65
9.8 Seed Verification Test Cases 66
9.9 Complaint System Test Cases 66
9.10 Security and Access Control Test Cases 67

List of Figures

3.1 High-level system architecture of the AgriVerify. 11

3.2 Data flow diagram for the seed verification pipeline 13

3.3 Simplified ER Diagram of the AgriVerify Platform 14

4.1 Sample images collected from ICAR Manipur. 16

5.1 Asynchronous model request processing flow. 30

5.2 Automated CI/CD pipeline and Serverless GCP Deployment Architecture. 31

6.1 Backend Clean Architecture Layers 34

6.2 Entity Relationship Diagram – AgriVerify System 39

6.3 JWT Authentication Flow 41

6.4 Seed Verification Workflow 42

7.1 Frontend Layered Architecture and Request Execution Flow 45

8.1 Architectural Workflow and Cloud Deployment Pipeline 52

9.1 Training and validation accuracy curves across 10 epochs. 56

9.2 Training and validation normalized loss curves across 10 epochs. . . 57

9.3 Precision, Recall, and F1 Score trajectories across 10 epochs. 57

9.4 Generalization gap (Train-Val difference) monitored against a 5% overfitting threshold. 58

9.5 Visual representation of Confusion Matrix 60

9.6 Comparison of validation accuracy profiles across different model architectures. 63

9.7 Comparison of validation loss profiles across different model architectures. 64
9.8 Platform landing page displaying value proposition and core features. 68
9.9 Multilingual selection interface supporting regional languages. . . . 68
9.10 Role selection interface for login. 69
9.11 Farmer registration form collecting demographic and location information. 69
9.12 Secure admin login requiring multi-factor authentication. 70
9.13 Farmer dashboard interface displaying verification statistics and history. 70
9.14 Farmer profile configuration interface with verification achievements. 71
9.15 User profile modification interface. 71
9.16 Mobile camera interface for uploading seed packaging images. . . . 72
9.17 Verification result indicating pure seed batch with morphological measurements. 72
9.18 Verification alert for counterfeit or low-quality seed batch. 73
9.19 Grievance registration interface for lodging seed quality complaints. 73
9.20 Farmer tracking interface for monitoring grievance status. 74
9.21 Agricultural officer dashboard presenting regional complaint metrics. 74
9.22 Filtered and prioritized grievance investigation queue for officers. . . 75
9.23 Officer grievance detail pane featuring user evidence and resolution controls. 75
9.24 Administrator real-time system performance telemetry dashboard. . 76
9.25 User administration and access control interface. 77
9.26 Analytical reporting dashboard showing quality trends and system statistics. 77
9.27 District-wise geospatial hot-spot mapping of counterfeit occurrences. 78

Chapter 1

Introduction

1.1 Background and Context

In states like Manipur, agriculture is central to the livelihood of most families, with paddy cultivation forming the cornerstone of both food security and the rural economy. In these communities, crop yields depend heavily on the quality of the seed sown. Even with proper water and soil management, low-quality or counterfeit seeds will inevitably result in poor germination rates, high vulnerability to local pests, and weak crop yields. Recently, agricultural markets in developing regions have seen a rise in the trade of counterfeit seeds.

These counterfeit seeds are often packaged and labeled to look exactly like high-yielding certified varieties, such as RC Maniphou-12 or CAU-R1, making them visually indistinguishable to the naked eye. Farmers generally only discover they have bought low-quality grain after sowing, by which point they have already lost their investment in fertilizers, labor, and precious planting windows. Traditional verification methods require sending samples to specialized agricultural laboratories for genetic or chemical testing. These tests are slow, expensive, and impractical for smallholder farmers who need to verify their seeds immediately before sowing.

1.2 Motivation

This project was prompted by the economic difficulties local farmers face when they unknowingly purchase low-quality or mislabeled seeds. In Manipur's rural farming communities, a single bad harvest due to counterfeit seeds can push a household into deep debt, disrupting local food supplies and weakening trust in local seed suppliers. There is a clear need for a simple, low-cost verification method

that agricultural workers and cooperative groups can use on the spot.

Recent developments in deep learning and computer vision offer a potential solution. Using convolutional neural networks, we can automate the visual analysis of grain patterns, textures, and shape variations to detect quality anomalies. The goal of this project is to take these machine learning techniques and package them into a lightweight web-based application. Since farmers in hilly districts like Senapati and Tamenglong often experience unstable 2G/3G mobile data connections, we wanted to design a highly optimized, low-bandwidth tool that runs in a standard mobile browser, providing immediate quality feedback without requiring expensive hardware.

1.3 Problem Statement

Before the planting season, farmers lack a fast and accessible tool to verify whether their paddy seeds match the quality standards of genuine certified varieties. Visual inspection by eye is unreliable, and lab tests are too slow.

To address this, we developed **AgriVerify**, an integrated web-based platform that uses deep learning to inspect paddy seed imagery and run text extraction on package labels. By cross-referencing visual grain features against known standards and tracking user-reported germination rates, the platform aims to flag potential counterfeit batches. This setup provides a real-time verification option that helps map out regional quality issues and assists agricultural officers in tracking low-quality seed distribution.

1.4 Objectives

The primary objective of this project is to build a practical, integrated software system for paddy seed verification. To achieve this, we outlined the following specific tasks:

1. To curate a localized dataset of paddy seed images and train a deep learning classifier to distinguish between high-quality seeds, damaged grains, and common contaminants.
2. To build a lightweight, mobile-responsive web frontend that allows users to upload seed photos for real-time model inference.
3. To implement a farmer feedback module where users can log actual germination rates and post-planting observations for their seed batches.

4. To establish a notification and dashboard system for agricultural officers to monitor reported anomalies and identify problematic seed batches.

1.5 Scope of the Project

This project focuses on the full-stack development and deployment of the AgriVerify web application. The work includes collecting and preprocessing digital images of local paddy varieties, training and optimizing a Convolutional Neural Network (CNN), and building the supporting database and API endpoints.

The system's analysis is strictly limited to non-destructive visual evaluation of seeds using standard digital cameras; it does not perform chemical, genetic, or biological laboratory tests. The final output is a working prototype that demonstrates how combining computer vision with crowdsourced farmer feedback can help identify substandard seed batches in the field.

Chapter 2

Literature Review

2.1 Seed Quality Assessment: Traditional Methods

Traditional seed quality assessment primarily relies on manual visual inspection and laboratory testing. Visual inspection evaluates characteristics such as seed size, shape, colour, and texture, but the process is subjective and may vary between inspectors. Laboratory methods, including Tetrazolium (TZ) and germination tests, provide more accurate results but are time-consuming, destructive, and require controlled conditions.

Advanced techniques such as Near-Infrared (NIR) spectroscopy and X-ray imaging offer non-destructive seed analysis and can detect internal defects. However, their high cost and requirement for specialized equipment limit their adoption to well-equipped testing laboratories. As a result, farmers and local inspectors often lack access to rapid, affordable, and reliable seed verification methods.

2.2 Deep Learning for Agricultural Image Detection

Convolutional Neural Networks (CNNs) have been applied to a wide range of agricultural image detection tasks over the past decade, including leaf disease detection [1], weed detection [2], and post-harvest quality assessment [3]. Their principal advantage over classical machine-learning approaches is the ability to learn spatial features directly from raw pixel data during training, eliminating the need for manually engineered feature extractors for each crop or defect type.

Applying CNNs to seed quality detection is a natural extension of this work. Paddy seeds often exhibit subtle variations in surface texture, colour, and visible defects between genuine and counterfeit batches. These differences are difficult to quantify

using handcrafted features but can be effectively identified by CNNs when trained on sufficiently large datasets. Furthermore, the computational requirements of a trained CNN during inference are modest, making server-side web deployment feasible. This is particularly beneficial for farmers in rural areas who may have limited access to advanced hardware but can access the system through mobile internet connectivity.

2.3 Transfer Learning and ResNet Architectures

Training a CNN from random initialisation requires on the order of tens of thousands of labelled examples per class to achieve stable convergence. Annotated seed image datasets of that scale do not yet exist for most crop varieties, and certainly not for the specific task of detecting counterfeits. Transfer learning is the standard response: a model pre-trained on ImageNet (1.28 million labelled images across 1,000 categories) is fine-tuned on the smaller domain-specific dataset. The early convolutional layers, which encode low-level features such as edges and textures, are retained; only the later task-specific layers are updated. In practice, this approach consistently reaches accuracy close to what a from-scratch model would achieve on a dataset roughly ten times larger.

ResNet architectures are well-suited to this fine-tuning scenario. The residual skip connections introduced by He et al. [4] allow gradients to propagate through very deep networks without vanishing, making it feasible to fine-tune models with 50 or more layers on datasets that would cause shallower architectures to overfit. ResNet-50 has become a common baseline in agricultural image detection, balancing parameter count ($\approx$25 million) against accuracy on standard benchmarks, with pre-trained weights publicly available. For this project, it provides a well-validated starting point that reduces both training time and the risk of overfitting on the available paddy seed dataset.

2.4 Farmer Feedback Systems

A detection model trained on a fixed dataset will degrade over time if counterfeiting practices evolve and no new training data are collected. Field-level feedback from farmers is one practical mechanism for capturing these shifts. When a farmer submits a seed batch for image-based verification and subsequently reports germination failure or abnormal yield, that report constitutes a labelled example, a model prediction paired with a ground-truth outcome, that can be incorporated into periodic retraining cycles.

Feedback also serves a more immediate diagnostic function. Aggregating reports across users can flag a suspect lot or supplier before the model itself is updated: three independent failure reports from the same district within a fortnight is a meaningful signal, regardless of what the image detector predicts. This rule-based alerting complements the model rather than replacing it. Sustaining farmer participation remains a practical difficulty; deployments in low-resource contexts consistently find that single-tap rating interfaces produce far higher response rates than free-text fields or structured forms [5].

2.5 Web-Based Agricultural Platforms

Deploying a trained model as a web service separates the computational workload from the end user's device. A farmer uploads a seed image through a browser; the server runs inference and returns a result. This architecture offers two concrete advantages over distributing a standalone mobile application: the model can be updated centrally without requiring client-side installations, and every prediction is logged in a structured format that supports monitoring and retraining.

For farmers in Manipur and comparable regions, browser-based access via a mid-range Android handset is more realistic than assuming a dedicated application installation. Keeping the frontend lightweight, avoiding heavy JavaScript bundles and large media assets reduces page-load times on 4G connections with variable signal quality. Containerised inference endpoints handle computationally intensive tasks without requiring a permanently running server, reducing operating costs during off-season periods when query volume is low.

2.6 Review of Related Work

Early automated seed detection systems employed Support Vector Machines (SVMs) and Random Forests using handcrafted features derived from shape measurements, colour histograms, and texture descriptors. Ni et al. [6] identified rice seed varieties using morphological parameters extracted from binary silhouettes, achieving 94% accuracy across five varieties under controlled lighting conditions. These methods perform adequately when variation within seed categories is low and imaging conditions are consistent, but they generalize poorly to photographs captured in real-world field environments.

Deep learning approaches have largely replaced handcrafted methods in recent studies. Liu et al. [7] applied VGG16 transfer learning for maize seed defect

detection, reporting 97.3% accuracy on a 12-category dataset. Xu et al. [8] used ResNet-50 for soybean quality assessment and found that fine-tuning from ImageNet weights outperformed training from scratch by approximately 8 percentage points when the training dataset contained fewer than 2,000 images per category. Both studies, however, focus on naturally occurring defects such as cracking, mould, and insect damage rather than intentional substitution.

A counterfeit seed is deliberately designed to resemble a genuine seed, making reliable identification significantly more challenging than detecting a damaged kernel. The visual differences between genuine and counterfeit seeds are often subtle, and the adaptive nature of counterfeiting means that counterfeit characteristics may evolve over time. To the best of our knowledge, no published study has directly evaluated CNN performance on verified counterfeit-versus-genuine paddy seed datasets, nor has any study integrated a structured farmer feedback mechanism into the seed verification process.

2.7 Summary and Research Gaps

The reviewed literature establishes that transfer-learned CNNs are effective for agricultural image detection, including seed quality assessment. Three gaps are directly relevant to this work. First, no published study has targeted intentional counterfeiting as distinct from natural quality defects; the two problems differ in that counterfeits may pass casual visual inspection and require the model to detect subtle discriminative cues. Second, existing models are evaluated on laboratory-quality images collected under controlled conditions; performance on photographs taken by farmers using mobile devices in variable lighting has not been reported. Third, no end-to-end system has been published that connects model inference to a structured feedback collection mechanism and uses that feedback iteratively for retraining.

This project addresses all three gaps. The proposed **AgriVerify** system is trained and evaluated on a counterfeit-versus-genuine paddy seed dataset. The web platform is designed to process images captured using mobile devices, making it accessible to farmers in real-world conditions. In addition, the farmer feedback module is integrated directly into the application, enabling users to report seed performance and field-level issues. These responses are stored in a structured format that can support future model improvement and system enhancement.

Chapter 3

System Design and Architecture

3.1 Overall System Architecture and Technology Stack

The AgriVerify platform is designed using a multi-tiered architecture based on the Clean Architecture model to ensure strict separation of concerns, decoupling of business logic from external frameworks and high maintainability [9]. The system comprises three primary layers: a Next.js frontend client web application, an ASP.NET Core backend Web API and a FastAPI deep learning inference microservice. These tiers interact to process seed validation requests, manage user authentication and persist data.

The backend API follows Onion Architecture principles, dividing responsibilities into four distinct concentric layers as detailed in Table 3.1.

Table 3.1: Backend Architectural Layers and Responsibilities

Architectural Layer	**Core Responsibilities and Encapsulated Components**
Domain Layer	Core domain entities (`User`, `VerificationHistory`), value objects and domain exceptions. Completely independent of external frameworks.
Application Layer	Orchestration logic (`VerificationService`, `UserService`), Data Transfer Objects (DTOs), mapping configurations and request validation rules.
Infrastructure Layer	External integration implementations, including database persistence via Entity Framework Core, Azure Cognitive Services and Blob storage managers.
API Layer	REST API controllers, request-response handling, JWT authentication middleware and unified exception filters.

Table 3.2 details the software development kits (SDKs), frameworks, libraries and cloud platforms employed across the frontend, backend and machine learning components.

Table 3.2: Technology Stack of AgriVerify

Component	**Framework / Library**	**Functionality / Purpose**
Frontend	Next.js 14 (TS)	Client interface, server-side rendering and API proxy routing [10].
	TanStack Query	Server state synchronization and asynchronous data caching [11].
	Zustand	Client-side authentication state management [12].
	Tailwind CSS	Styling system and component customization [13].
Backend	ASP.NET Core 8	Orchestration Web API, routing and controller layer [14].
	Entity Framework Core	Object-Relational Mapping (ORM) for SQL Server database access.
	Azure Cognitive Services	Optical Character Recognition (OCR) and Custom Vision packaging verification.
	Azure OpenAI Service	Natural language synthesis for seed verification reports.
ML	PyTorch 2.1	Deep learning framework for neural network training [15].
	ResNet-50	Pre-trained CNN backbone for seed quality assessment [4].
	OpenCV	Preprocessing and morphometric DUS parameter analysis [16].
	FastAPI	Serverless microservice API on GCP Cloud Run [17].

3.2 Functional and Non-Functional Requirements

System requirements are categorized into functional specifications, defining the core operations and roles within the application, and non-functional specifications, outlining quality attributes and system constraints. These requirements ensure that the platform meets both agricultural user needs and architectural standards.

The functional requirements (Table 3.3) define the core capabilities across different system actors (farmers, agricultural officers and administrators). Concurrently, the non-functional requirements (Table 3.4) dictate operational performance, security and portability parameters, ensuring standard enterprise scalability and reliability.

Table 3.3: System Functional Requirements

ID	**Module / Role**	**Functional Specification**
FR-01	Authentication	User registration and authentication via JWT for farmers, officers and administrators.
FR-02	Image Submission	Multi-modal submission (packaging and seed image) for real-time verification.
FR-03	AI Inference	Execution of Custom Vision, OCR text extraction and ResNet-50 DUS profiling.
FR-04	Feedback System	Reporting mechanism allowing farmers to submit grievances on counterfeit seed batches.
FR-05	Case Management	Dashboards for officers to track, update and resolve seed complaints.
FR-06	Risk Analytics	Automated aggregation of regional complaints in a Batch Risk Registry for analytics.

Table 3.4: System Non-Functional Requirements

ID	**Attributes**	**Specification**
NFR-01	Performance	Target end-to-end inference latency under 3.0 seconds through parallel service calls.
NFR-02	Scalability	Stateless API design facilitating horizontal scaling and microservice containment.
NFR-03	Security	HTTPS communication, JWT session control, role-based authorization (RBAC) and hashed secrets [18].
NFR-04	Reliability	Atomic database operations and soft-delete implementation for data auditability.
NFR-05	Usability	Responsive UI designed for mobile and desktop screens, optimized for low-bandwidth environments.

3.3 High-Level System Architecture

The system architecture is structured as a three-tier model: the Client Layer, the Backend Orchestration Tier and the Cloud AI and Data Infrastructure Tier. As illustrated in Figure 3.1, client applications do not interact directly with the backend API; rather, all requests are routed through a server-side proxy in the Next.js frontend web tier. This proxy injects JSON Web Tokens (JWT) for authentication and forwards requests to the ASP.NET Core 8 Web API. The API layer orchestrates calls to the persistent data store (SQL Server), cloud storage and AI inference services.

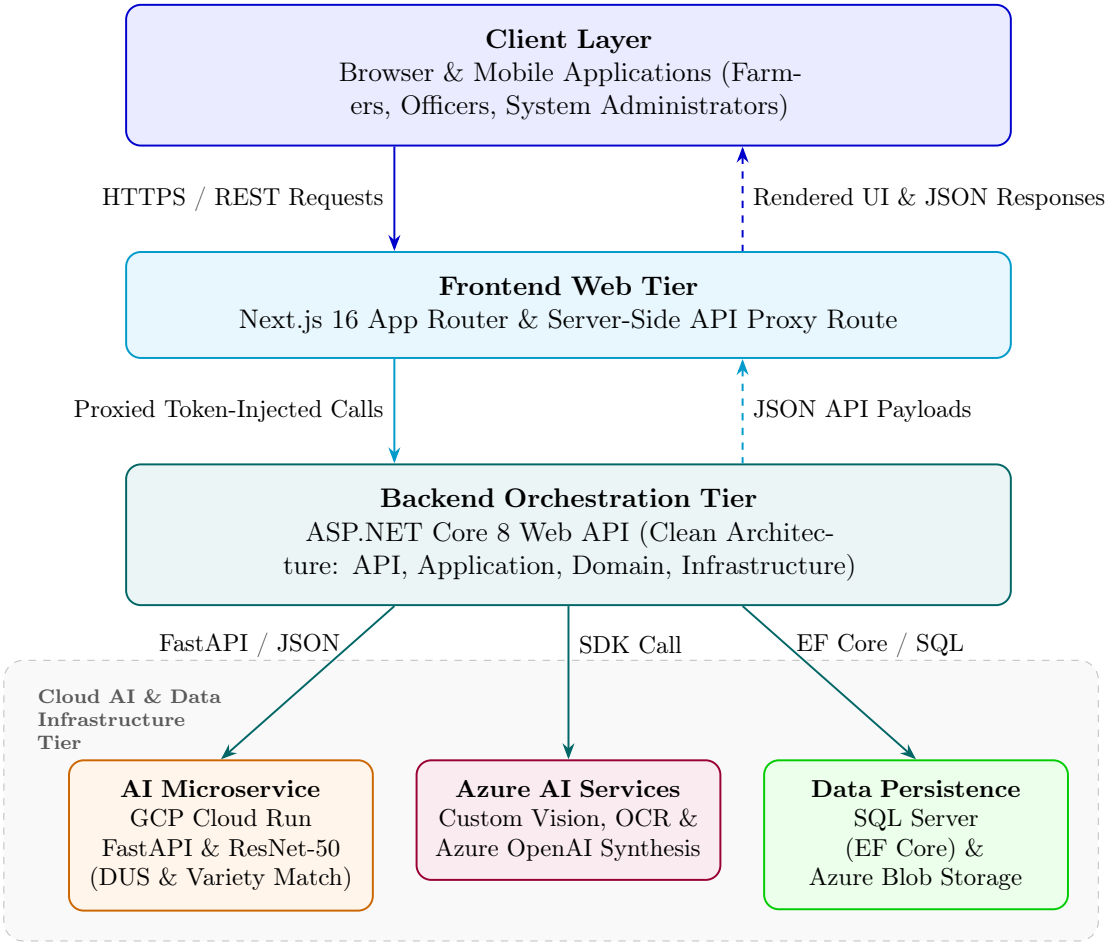

Figure 3.1: High-level system architecture of the AgriVerify.

3.4 System Data Flow Pipeline

The step-by-step verification pipeline and data transmission flow are depicted in Figure 3.2. The execution sequence is structured as follows:

1. **Submission:** The client uploads multi-part image data representing seed

packaging and physical seeds.

2. **Routing and Proxying:** The Next.js API proxy interceptor validates user sessions, appends a secure bearer token, and executes an HTTP POST request to the backend.

3. **Parallel Execution:** The ASP.NET Core orchestration engine coordinates concurrent asynchronous requests using asynchronous task-based execution.

4. **Feature Extraction:**

   - Custom Vision classifies the packaging (Genuine, Counterfeit, Suspicious).
   - Azure AI Vision extracts text strings via OCR to identify seed batch codes.
   - The GCP Cloud Run ResNet-50 service determines 17 morphological parameters.

5. **Synthesis and Persistence:** The OpenAI integration consolidates these individual model predictions into a coherent diagnostic summary, which is stored in the relational database via Entity Framework Core before the final JSON payload is returned to the user interface.

3.5 Database Entity-Relationship Model

The relational database schema is modeled to ensure data consistency, minimize redundancy, and support real-time querying. Figure 3.3 defines the entity relations, key fields, and cardinalities within the AgriVerify database. The core architectural design decisions are detailed as follows:

- **User and Security Tracking:** The `User` entity supports role-based access control and maintains a 1-to-many relationship with `RefreshToken` to secure active sessions.
- **Verification History:** The `VerificationHistory` table preserves all client evaluations and maintains a soft-delete status (`IsDeleted`) to enforce an immutable audit trail.
- **Feedback and Case Resolution:** When a seed dispute is identified, a `ProductComplaint` record is initiated. This entity links to the executing officer, records ongoing investigations via `ComplaintAction`, and syncs batch metrics with the `BatchRiskRegistry`.

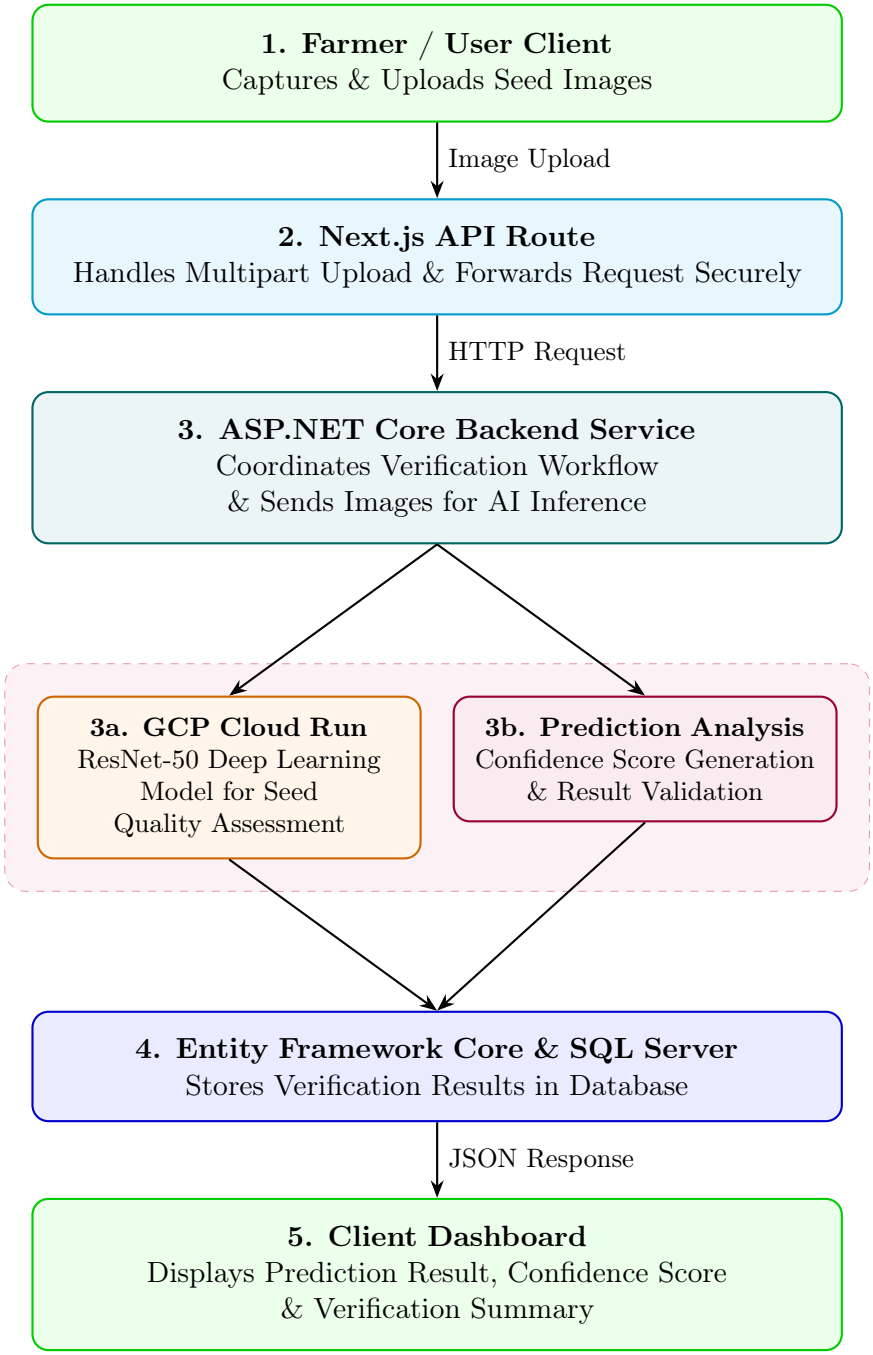

Figure 3.2: Data flow diagram for the seed verification pipeline

- **Batch Analytics:** The `BatchRiskRegistry` dynamically compiles aggregation data (complaint counts, severity indices, and regional distributions) to optimize query performance for system dashboards.

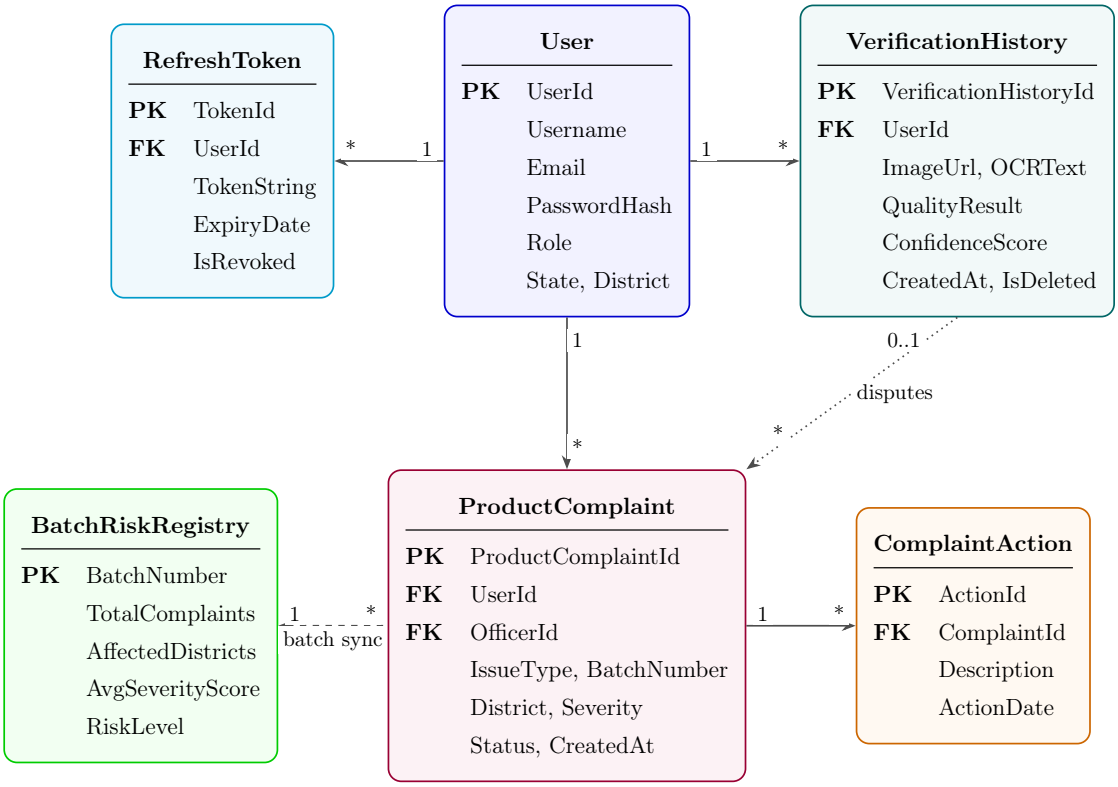

Figure 3.3: Simplified ER Diagram of the AgriVerify Platform

Chapter 4

Deep Learning Model for Paddy Seed Quality Assessment

4.1 Dataset Description

4.1.1 Data Sources and Collection (Kaggle)

We built our training dataset by combining two sources from Kaggle. The first is a publicly available dataset, and the second is a custom collection we put together in partnership with India's Council of Agricultural Research (ICAR) facility in Manipur. By bringing these two sources together, we got a larger and more realistic dataset—one that includes seed images captured in different ways and from different parts of India. This diversity is crucial because it helps the model learn to recognize seeds regardless of lighting conditions or camera setup.

4.1.1.1 Public Source

The first source is the Paddy Seeds Quality Classification dataset published on Kaggle by Ayyan Arkadalkani:

`https://www.kaggle.com/datasets/ayyanarkadalkani/paddy-seeds-quality-classification-dataset`

This dataset contains 1,213 labeled images split into three categories: Genuine, Suspicious, or Low Quality seeds. The images take up about 183 MB of storage. These images were captured under many different lighting conditions and backgrounds, which made them a solid foundation for our model to learn general patterns about what different seed qualities look like.

4.1.1.2 Custom Source

The second source is a dataset we collected directly from ICAR's Manipur laboratory. ICAR researchers took photographs of paddy seeds under controlled lab conditions, focusing on the specific varieties grown in northeastern India. This custom collection includes 350 images occupying about 24.7 MB of storage. We've published this dataset on Kaggle as an open resource so other researchers can use it:

`https://www.kaggle.com/datasets/nilambarelangbam/paddy-seed-images-collected-from-icar-manipur`

Figure 4.1: Sample images collected from ICAR Manipur.

When we combined both datasets, we ended up with 1,563 images that represent a wider variety of seed conditions than either dataset alone. This combined dataset better reflects what farmers and agricultural officers actually encounter in the field across different regions of India.

4.1.2 Dataset Statistics and Class Distribution

After merging the two datasets, we had a total of 1,563 paddy seed images divided into three quality categories. A detailed breakdown of the datasets, their image counts, storage sizes, and public Kaggle repositories is summarized in Table 4.1, while the class-wise distribution of these images is provided in Table 4.2.

Table 4.1: Dataset Source Summary

Source	**Images**	**Size**	**Kaggle Identifier**
Paddy Seeds Quality Classification (Ayyan Arkadalkani)	1,213	∼183 MB	`ayyanarkadalkani/paddy-seeds-quality-classification-dataset`
ICAR Manipur Custom Dataset (Our Collection)	350	∼23.7 MB	`nilambarelangbam/paddy-seed-images-collected-from-icar-manipur`
Combined Total	**1,563**	**∼208 MB**	—

Table 4.2: Class-wise Distribution of the Combined Dataset

Class	**Number of Images**	**Percentage (%)**
Genuine Paddy Seeds	∼750	∼48.0
Suspicious Paddy Seeds	∼510	∼32.6
Low Quality Seeds	∼303	∼19.4
Total	**1,563**	**100.0**

From Table 4.2 we observed that, the dataset we have contain more Genuine seed images than Suspicious or Low Quality ones which makes sense because in real life, most seeds are genuine. However, this imbalance could cause the model to become biased toward recognizing genuine seeds and miss the problematic ones. To fix this, we used a technique called weighted sampling during training: when building each training batch, we ensured each quality category contributed fairly, regardless of how many examples we had of each type.

4.1.3 Train/Validation/Test Split Strategy

We divided the 1,563 images into three separate groups: one for training, one for validation, and one for testing. A key principle we followed was stratified splitting, which means we made sure each group had roughly the same mix of Genuine, Suspicious, and Counterfeit seeds as the overall dataset. The proportions, approximate image counts, and primary objectives of each split are summarized in Table 4.3. This partitioning prevents any group from accidentally having too many of one type, which could give us misleading results.

Table 4.3: Dataset Split Statistics

Split	**Proportion**	**Approx. Images**	**Purpose**
Training Set	80%	∼1,250	Used to teach the model by adjusting its internal weights
Validation Set	10%	∼157	Used to tune settings and watch for the model learning too much from training data
Test Set	10%	∼156	Held completely separate to give a final, unbiased measure of how well the model works

Table 4.3 presents the data split statistics used in this study. The training set is used to train the model, allowing it to learn from the images and adjust its internal parameters to improve model performance. The validation set serves as a quality control mechanism; after each training epoch, the model is evaluated on this set to monitor its performance and detect potential overfitting, where the model memorizes the training data rather than learning generalizable patterns. The test set remains completely unseen during the training and validation phases. It is used only once at the end of the process to provide an unbiased and reliable evaluation of the model's performance on previously unseen images.

4.1.4 Data Preprocessing and Augmentation

Before showing images to the neural network, we prepared them using a standard pipeline. All images were resized to 224×224 pixels—this is the standard size that ResNet50 (our neural network architecture) expects. We then adjusted the color values using ImageNet statistics (numbers from a large, well-known image dataset):

$$\mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225] \tag{4.1}$$

This normalization step is important because the pre-trained model we started with was trained on images normalized this way, so we keep the same approach for

consistency.

To make the model more robust and help it learn from our limited training data, we applied six data augmentation techniques during training. Think of augmentation as artificially creating new training examples by tweaking the original images in realistic ways:

1. **Random Resized Crop** We randomly crop a portion of the image and resize it to 224×224 pixels. This simulates seeds being photographed from different distances and positions within the frame.

2. **Horizontal Flip** We randomly flip images left-to-right about half the time. Seeds have a natural bilateral symmetry, so this helps the model learn that a flipped seed is still the same seed.

3. **Vertical Flip** We randomly flip images top-to-bottom about half the time, adding more variation in how the seeds appear.

4. **Random Rotation** We rotate images randomly by up to $\pm 30°$ (30 degrees in either direction). Real seed photos might not be perfectly aligned, so this prepares the model for that reality.

5. **Translation/Random Shift** We randomly shift the image content left, right, up, or down. This prevents the model from relying on the seed always appearing in the center of the photo.

6. **Colour Jitter** We randomly adjust brightness, contrast, color saturation, and hue. Since real agricultural photos are taken under different lighting conditions, this helps the model handle that variation.

4.2 Model Architecture

4.2.1 ResNet50 Backbone (25.6M Parameters)

We chose ResNet50 as the foundation of our detection model. This neural network has 50 layers and about 25.6 million parameters (adjustable settings inside the network). What makes ResNet special is its use of "skip connections" shortcuts that let information flow directly around groups of layers. This design makes it much easier to train very deep networks because it prevents the gradient (the training signal) from fading as it travels backward through the network.

Rather than training from scratch, we started with weights that were pre-trained on ImageNet, a massive dataset of millions of everyday images. This pre-trained

starting point is like giving the model a head start: it already knows how to recognize edges, textures, and shapes, so we only need to teach it the specific patterns that distinguish different seed qualities.

4.2.2 Custom Model Head

On top of the ResNet50 backbone, we added a custom section designed specifically for seed quality assessment. This section takes the features learned by ResNet and processes them into a final decision about whether a seed is Genuine, Suspicious, or Low Quality. Here's what this custom section does:

1. Compresses the spatial information into a single summary vector (Global Average Pooling)
2. Passes it through a fully connected layer with 512 hidden units
3. Applies Batch Normalisation to stabilize the values
4. Uses ReLU activation to introduce non-linearity
5. Applies Dropout (randomly ignoring 50% of values) to reduce overfitting
6. Outputs 3 final scores (one for each seed quality) with softmax to convert them to probabilities

4.2.3 Anti-Overfitting Techniques

Overfitting is the enemy of machine learning—it's when a model memorizes the training data instead of learning generalizable patterns. We used several techniques to prevent this:

- **Dropout**—During training, we randomly "turn off" 50% of neurons in the model head. This forces the network to learn redundant representations, so it can't rely on a few specific neurons. Think of it like training a team where you randomly swap out players—the team learns to work together, not depend on one star player.
- **Batch Normalisation**—After each layer, we normalize the values to keep them in a genuine range. This stabilizes training and acts as a natural regularizer.

- **Label Smoothing**—Instead of forcing the network to be 100% confident about each prediction, we "soften" the targets slightly (using $\epsilon = 0.1$). This prevents the model from becoming overconfident and improves generalization.
- **Weight Decay**—We penalize large weights during training, keeping the model's parameters from growing too large. A smaller model is typically more generalizable.
- **Data Augmentation**—As described earlier, we artificially expand the training data with slight variations, giving the model more diverse examples to learn from.
- **Early Stopping**—We continuously monitor performance on the validation set. If it starts getting worse, we stop training immediately and use the best weights we've found so far.

4.3 DUS Parameter Integration

4.3.1 Feature Extraction—17 Physical Parameters

Beyond just classifying seeds as Genuine, Suspicious, or Low quality, we wanted our model to extract physical measurements that agriculture experts understand. The standard set of measurements in agriculture is called DUS—Distinctiveness, Uniformity, and Stability. We programmed our model to extract 17 of these physical parameters from each seed image:

- How long and how wide each seed is
- The ratio of length to width
- The perimeter and total area of the seed
- How circular the seed is (circularity) and how solid it is (solidity)
- How elongated the seed is (eccentricity)
- Mathematical descriptions of how the seed's mass is distributed
- Hu moments—a special mathematical descriptor of seed shape that works the same way whether the seed is rotated, scaled, or shifted in the image

These 17 parameters bridge the gap between what the neural network learns (visual patterns) and what agricultural experts care about (standard physical

measurements). This makes our system more transparent and trustworthy to end users.

4.3.2 Pixel-to-Physical Unit Conversion

When we measure seeds in a photograph, the measurements are in pixels, which depends on the camera and how far away it was. To convert pixel measurements to real-world millimetres, we include a reference object (a ruler with known dimensions) in each photograph. We then calculate:

$$\text{scale} = \frac{\text{reference\_length\_mm}}{\text{reference\_length\_pixels}} \tag{4.2}$$

This scale factor is applied to all measurements, so when we report that a seed is 6.2 mm long, it's truly 6.2 millimetres—not dependent on camera distance or image resolution.

4.3.3 Variety Matching (12 Reference Varieties)

We created a reference database of 12 standard paddy seed varieties used in India, based on official ICAR documentation. For each variety, we computed the typical values of all 17 physical parameters from multiple certified seed samples. When a new seed image is analyzed, we compare its 17 parameters against these 12 reference profiles using a simple distance calculation:

$$d_i = \sqrt{\sum_{j=1}^{17} (p_j - r_{i,j})^2} \tag{4.3}$$

Here, p_j is one of the 17 measurements from the new seed, $r_{i,j}$ is the standard value for variety i, and d_i tells us how different the new seed is from variety i. The variety with the smallest distance is the best match—this is the variety the seed most closely resembles. We report the top 3 matches so users can see alternatives if needed.

4.4 Training Pipeline

4.4.1 Phase 1—Model Head Training (Epochs 1–8)

We trained the model in two phases. In Phase 1 (the first 8 training cycles), we froze the ResNet50 backbone—meaning its weights didn't change—and only trained the custom model head we added on top. Think of this like keeping a well-established expert's knowledge fixed and just training a new student to interpret their findings. We used a learning rate of 0.001, which controls how big the weight updates are. A learning rate that's too high causes instability; too low and training is slow. The batch size (number of images processed before updating weights) was set to 32.

At the end of each of these 8 epochs, we checked performance on the validation set to make sure the head was learning properly and not starting to overfit.

4.4.2 Phase 2—Full Fine-Tuning (Epochs 9–15)

In Phase 2 (epochs 9–15), we unfroze the entire network and trained everything end-to-end. We reduced the learning rate by 10 times to 0.0001—a smaller rate because we're now adjusting a fully-trained backbone and don't want to destroy what it's already learned. This phase refined the backbone's features to specialize better for seed detection.

We monitored validation performance carefully and stopped training at epoch 15 when we noticed the validation loss starting to increase (a sign the model was beginning to overfit). We then loaded the best weights we'd found during the entire training run—these became our final model.

4.4.3 Hyperparameter Configuration and Optimiser

A comprehensive summary of the optimizer, learning rates, regularizers, and general training configurations is provided in Table 4.4:

We chose the AdamW optimiser because it's smart about adjusting the learning rate for different parameters and handles weight decay better than older methods.

Table 4.4: Hyperparameter Configuration and Training Settings

Hyperparameter	**Value**	**Notes**
Optimiser	AdamW	Smart learning rate that adapts during training
Phase 1 Learning Rate	0.001	Used while training only the head
Phase 2 Learning Rate	0.0001	Used for full network fine-tuning (10 times smaller)
Weight Decay (L2)	0.0001	Penalizes large weights to keep the model simple
Batch Size	32	Number of images processed before updating weights
Loss Function	Cross-Entropy + Label Smoothing	Prevents overconfidence with $\epsilon = 0.1$
LR Scheduler	StepLR	Reduces learning rate further after 8 epochs
Gradient Clipping	Max norm = 1.0	Prevents unstable training due to large gradients
Early Stopping Patience	10 epochs	Stops if validation loss doesn't improve for 10 epochs
Input Resolution	224×224 pixels	Standard size for ResNet50
Total Epochs	15	Stopped when overfitting began

4.4.4 Model Export Formats

After training finished, we saved the model in multiple formats to ensure compatibility with different environments. The exported files, including their sizes and target use cases, are listed in Table 4.5.

Table 4.5: Model Export Formats

Format	**Extension**	**Size**	**Use Case**
PyTorch Native	`.pth`	∼98 MB	The original format; used by FastAPI for serving
TorchScript	`.pt`	∼97 MB	For production servers and systems using C++
ONNX	`.onnx`	∼98 MB	For cross-platform use and systems not using PyTorch
Configuration	`.json`	∼2 KB	Metadata about the model, class labels, and settings
DUS Reference DB	`.csv`	<1 MB	The 12 reference seed varieties for matching

4.5 Model Comparison and Selection

A performance comparison among all four models is shown in table 4.6. Before committing to ResNet50, we tested four popular neural network architectures on the same dataset, using the same training approach, to see which performed best. We compared MobileNetV2, DenseNet121, and GoogleNet, with the comparative results detailed in Table 4.6.

Table 4.6: Architecture Comparison on Test Set

Model	**Accuracy**	**Precision**	**Recall**	**F1 Score**	**Time (min)**
ResNet50	99.57%	99.57%	99.57%	99.57%	16.4
DenseNet121	99.57%	99.57%	99.57%	99.57%	15.0
GoogleNet	99.57%	99.57%	99.57%	99.57%	12.2
MobileNetV2	99.13%	99.15%	99.13%	99.13%	8.5

As from the table 4.6 we observed that ResNet50, DenseNet121, and GoogleNet all achieved the same 99.57% accuracy, demonstrating that our dataset is rich enough that all three architectures can learn to classify seeds nearly perfectly. MobileNetV2 achieved slightly lower accuracy at 99.13% but trained much faster—in just 8.5 minutes compared to 16.4 minutes for ResNet50.

We chose ResNet50 for three practical reasons:

1. Its design with skip connections is well-understood in the research community, making it easier to diagnose training issues and fine-tune behavior.
2. It has excellent support across different deployment platforms—PyTorch, TorchScript, ONNX, and more—giving us flexibility in where and how we deploy it.
3. Most published agricultural image detection research uses ResNet50, so our results are directly comparable to other teams' work, helping the scientific community benchmark progress.

The extra 4.4 minutes of training time is a small price to pay for these advantages, especially since training only happens once.

4.6 Final Model Performance Summary

After completing both training phases, we evaluated our final ResNet50 model on the test set—the images we'd held back and never shown the model during training. The final evaluation metrics are compiled in Table 4.7, and a detailed summary of the training trajectory and learning stability is provided in Table 4.8.

Table 4.7: Final Test Set Performance—ResNet50

Metric	**Value**	**Status**
Test Accuracy	99.57%	Far exceeds industry standard (85–90%)
Precision	∼0.99	Outstanding—almost no false positives
Recall	∼0.99	Outstanding—catches almost all cases
F1 Score	∼0.99	Excellent—perfectly balanced performance
Train-Validation Gap	<5%	Great generalization—model isn't overfitting

From Table 4.7 we observed that, the final test accuracy of 99.57% significantly exceeds the typical industry benchmark of 85–90% for automated seed quality detection/assessment. The near-perfect precision, recall, and F1-score indicate highly reliable performance across all seed quality categories. These results demonstrate that the proposed system is suitable for practical deployment and can support accurate seed quality assessment in agricultural applications. “‘

Table 4.8 summarizes the key observations recorded during the training phase of the ResNet50 model. Monitoring training and validation accuracy along with loss values helps evaluate convergence, learning stability, and generalization capability. The observations presented below indicate that the model learned effectively, maintained stable performance throughout training, and benefited from the use of early stopping.

Table 4.8: Training Behaviour Summary

Observation	**Detail**
Initial Convergence	The model started at about 86% accuracy and quickly improved to near its best performance within the first few epochs
Loss Trajectory	The loss (error) steadily decreased from about 1.0 to nearly 0.0 across all 15 epochs
Mid-training Stability	After the initial improvement phase, training was smooth and consistent with no wild fluctuations
Overfitting Control	The gap between training accuracy and validation accuracy never exceeded 5%, indicating the model learned general patterns rather than memorizing training data
Training Stopped at	Epoch 15—we detected the first signs of overfitting and stopped immediately, preserving the best weights we'd found

The observations in Table 4.8 show that the training process was stable and effective. The model converged quickly, maintained high accuracy throughout training, and exhibited a small gap between training and validation performance. Early stopping further helped preserve the best-performing model while minimizing the risk of overfitting.

Chapter 5

Model Serving API

Deploying a trained model as a live service introduces a different set of problems than training it. Model development focuses on architecture and accuracy; the serving layer determines response latency, concurrent throughput and uptime under real load [9]. In AgriVerify, the PyTorch ResNet-50 model from **Chapter 4** runs inside a dedicated inference microservice built specifically for this purpose.

This chapter describes the design of that microservice, implemented with FastAPI and Uvicorn [19]. It covers the asynchronous request lifecycle, the tensor normalization pipeline, the API endpoint contracts, and the containerised deployment approach using Docker multi-stage builds on GCP Cloud Run [20].

5.1 API Architecture and Request Orchestration

5.1.1 FastAPI and ASGI Serving Stack

The inference microservice runs on FastAPI atop the Uvicorn ASGI runtime. FastAPI was chosen for its native async I/O support, Pydantic-based request validation and automatic OpenAPI schema generation [19]. Uvicorn's `uvloop`-backed event loop handles large numbers of concurrent connections with low memory overhead.

The stack separates network deserialization, Pydantic validation, OpenCV preprocessing and PyTorch execution into distinct layers. Computationally expensive inference operations run in dedicated thread pools and do not block the ASGI event loop.

5.1.2 End-to-End Model Request Flow

As illustrated in Figure 5.1, an inference request begins when the ASP.NET Core backend sends a multipart POST containing a seed image. FastAPI validates the MIME type and payload boundaries, then decodes the byte stream into an OpenCV matrix in memory.

From there, two operations run in parallel: a PyTorch inference engine with ResNet-50 weights pre-loaded to avoid cold-start delays[4], [21], while OpenCV contour routines extract 17 DUS (Distinctness, Uniformity, and Stability) morphological parameters [22]. The results are aggregated, scored against reference cultivars and returned as JSON.

The sequential execution steps of this request flow are defined as follows:

1. **Request:** The FastAPI endpoint receives the image payload via a multipart POST request from the backend proxy.
2. **Preprocessing:** The image is decoded into an OpenCV matrix, resized to 224×224 pixels, and normalized using the standard ImageNet channel moments.
3. **Model Inference:** The pre-loaded ResNet50 model runs inference in a background thread pool to generate raw seed quality assessment.
4. **Morphological Post-processing:** Simultaneously, OpenCV contour analysis extracts the 17 DUS physical measurements, and the system performs variety matching against the 12 reference profiles.
5. **Response:** The API aggregates the model predictions, confidence levels, variety distance scores and DUS metrics into a structured JSON payload returned to the client.

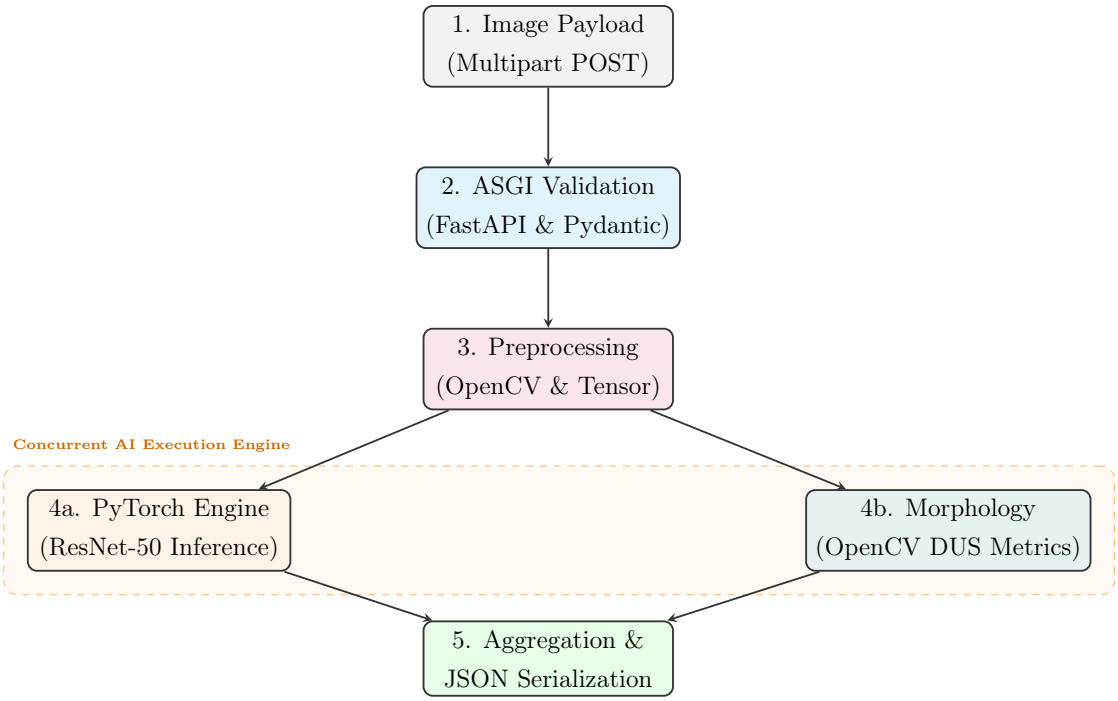

Figure 5.1: Asynchronous model request processing flow.

5.2 API Specification and Endpoint Contracts

The microservice exposes a tightly constrained RESTful API. Tables 5.1 and 5.2 define the schema contracts for communication with the backend orchestrator.

Table 5.1: API Specification for the `POST /predict` Model Inference Endpoint

Attribute	**Endpoint Constraint / Schema Definition**
Route & Auth	`POST /predict` | Requires `Authorization: Bearer <token>`
Payload	`multipart/form-data` (File: `image`). Types: JPEG/PNG. Max size: 10MB.
200 OK Response	`{"predicted_label": "Genuine", "confidence": 0.98, "inference_ms": 42.8, "dus_parameters": {"length_mm": 8.4, "aspect_ratio": 3.1}}`
Error Codes	**400**: Malformed payload | **413**: File too large | **415**: Invalid MIME type

5.3 Containerisation and Deployment Architecture

The service uses a multi-stage Docker build to balance reproducibility against image size [20]. The **Builder Stage** compiles Python binaries (`.whl`) with `gcc`. The **Runtime Stage** copies those pre-compiled binaries into a minimal, security-hardened Debian base (`python:3.10-slim`), discarding the build toolchain

Table 5.2: API Specification for the `GET /health` Liveness Endpoint

Attribute	**Endpoint Constraint / Schema Definition**
Route	`GET /health` (Unauthenticated liveness/readiness probe)
Probing Logic	Confirms Uvicorn ASGI event loop and verifies ResNet-50 weights are loaded in memory.
200 OK	`{"status": "online", "model_loaded": true, "device": "cuda:0"}`
503 Error	Service unavailable (model weights corrupted or GPU allocation failure).

entirely. The final image is approximately 850MB down from over 3GB with a correspondingly smaller attack surface. The automated containerization, versioning and deployment pipeline is illustrated in Figure 5.2.

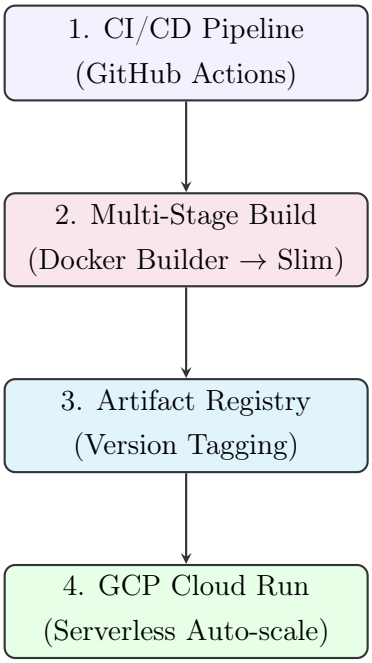

Figure 5.2: Automated CI/CD pipeline and Serverless GCP Deployment Architecture.

The image is deployed to GCP Cloud Run [23]. Cloud Run manages HTTPS termination and load balancing, and scales to zero when idle, so infrastructure costs track actual query volume rather than reserved capacity.

5.3.1 CI/CD Deployment Pipeline

The automated build and release lifecycle is managed via a continuous integration and continuous deployment (CI/CD) pipeline implemented with GitHub Actions. Whenever updates are merged into the repository's production branch,

the pipeline executes the following stages:

- **Build Validation:** Runs linter checks and automated tests to ensure code quality.
- **Docker Compilation:** Triggers a multi-stage Docker build, generating a security-hardened and size-optimized production container.
- **Registry Storage:** Authenticates with Google Cloud and pushes the compiled image to the secure GCP Artifact Registry with versioned tags.
- **Serverless Deployment:** Deploys the container to GCP Cloud Run, utilizing serverless auto-scaling to dynamically adjust resource allocation based on request volumes.

5.3.2 Live Production API Endpoint

To demonstrate the functionality of the serving layer under real-world conditions, the API is exposed as a live web service. Other client applications and services in the AgriVerify platform (such as the ASP.NET Core backend proxy) communicate with this service at:

`https://resnet-api-297419987783.asia-south1.run.app/`

Chapter 6

Backend System Architecture and Implementation

Our backend application coordinates request routing, database state, and downstream machine learning inference. We built the service using ASP.NET Core on .NET 8, organizing the code around Clean Architecture principles. This separation ensures that core business logic remains independent of external databases and third-party APIs.

Isolating the database configuration and external API adapters from the domain entities allowed us to develop the system incrementally. For example, during testing, we could swap out the cloud-based SQL Server configuration for an in-memory database instance without altering the service logic that runs the seed verification rules.

6.1 Architectural Design

The backend uses four distinct projects corresponding to the layers in Figure 6.1. Dependencies flow inward, meaning inner projects cannot reference outer ones. This compile-time constraint prevents database or presentation concerns from leaking into the core logic.

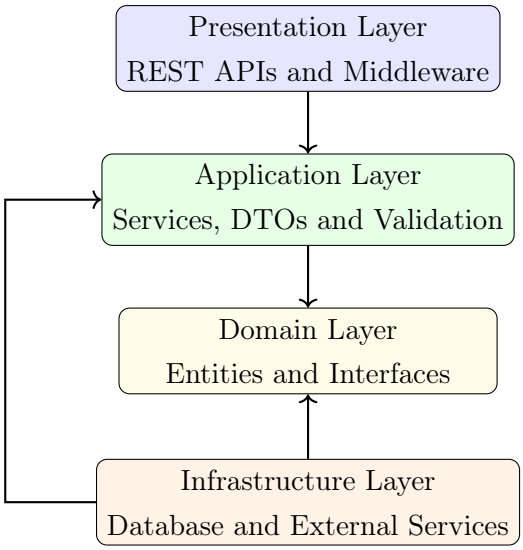

Figure 6.1: Backend Clean Architecture Layers

This physical separation simplifies testing. When writing unit tests for our verification logic, we mocked the repository interfaces and simulated image analysis runs offline without needing active connections to Azure or SQL Server. It also helps with deployment; we can scale the API presentation layer on serverless containers during peak harvesting seasons while leaving the underlying database server on a fixed tier to control costs.

6.2 Technology Stack

The backend uses specific libraries and frameworks to support request handling, user session management, and external service client interfaces. The core components of this technology stack are outlined in Table 6.1.

Choosing ASP.NET Core was driven by its Kestrel web server performance and built-in asynchronous pipeline. Handling multi-part form data uploads for 300KB images requires non-blocking I/O thread management, otherwise, simultaneous uploads from multiple farmers would exhaust the server's thread pool. For data persistence, Entity Framework Core serves as our database adapter. Using code-first migrations automated schema updates as we added fields for DUS morphological properties, avoiding the need to run manual SQL alter scripts across development and production environments.

Table 6.1: Backend Technology Stack

Technology	**Purpose**
ASP.NET Core Web API	Backend API framework
C# and .NET 8	Backend programming environment
Entity Framework Core	Database ORM and migrations
SQL Server	Relational database management
JWT Authentication	Secure user authentication
ASP.NET Core Identity	User and role management
Azure OCR	Text extraction from seed packets
Azure OpenAI	AI-generated explanations
Azure Blob Storage	Cloud image storage
FluentValidation	Input validation
AutoMapper	DTO mapping
Swagger	API documentation
Serilog	Request logging and monitoring

6.3 Project Structure

We organized the backend source code into distinct physical folders to maintain responsibility boundaries across the architecture, as detailed in Table 6.2.

Table 6.2: Backend Project Structure

Folder / Layer	**Purpose**
Domain	Business entities and interfaces
Application	Services, DTOs and validation
Infrastructure	Database and cloud integrations
Presentation	API controllers and middleware
Persistence	Repositories and migrations
Services	AI and Azure integrations
Configurations	Application configuration files
Middleware	Authentication and exception handling

By structuring the solution into separate projects, the .NET compiler enforces our architectural boundaries. The Domain project compile target has zero dependencies. The Application project, containing the business interface definitions, references only Domain. Concrete implementations—like EF Core context classes, Azure SDK integrations, and controller routes—reside in the Infrastructure and Presentation projects respectively, preventing presentation details from dictating business structures.

6.4 Domain Layer

The Domain Layer contains the application's core data models and interface contracts. This project has zero dependencies on external storage libraries, web frameworks, or ORM tools, ensuring that our agricultural verification rules remain isolated from infrastructure changes.

6.4.1 Core Entities

Our domain models map the physical elements of the seed verification workflow to structured data definitions. The `User` base class integrates with ASP.NET Core Identity to handle authentication, but we extended it with a one-to-one relationship to a `Profile` entity storing regional details (like District and Phone) to support geographic query filtering by agricultural officers. The physical components are modeled through `Seed` (defining reference cultivars) and `SeedBatch` (representing physical seed batches distributed by manufacturers).

When a verification scan is processed, the system creates a `DetectionResult` containing model labels, confidence levels, and physical parameters. If a farmer reports an issue with a seed batch, a `Complaint` entity is generated to track the investigation details. Lastly, we log system actions through the `AuditLog` entity to maintain a clear record of administrative and verification changes.

6.4.2 Interfaces

Rather than allowing the business logic to directly call SQL Server or Azure services, we defined interfaces within the Domain layer. These include repository abstractions like `IRepository<T>` and functional service interfaces like `IVerificationService` and `IComplaintService`. During unit testing, this allowed us to substitute the live cloud integrations with mock classes, meaning we could test the verification routing and database mappings locally without paying for Azure consumption or relying on active internet access.

6.5 Application Layer

The Application Layer coordinates the business workflows and transaction boundaries of the platform.

6.5.1 Application Services

The Application layer houses the coordinate logic for the system's workflows. The `VerificationService` handles the steps of the analysis pipeline: validating file parameters, calling the FastAPI model service, uploading the raw image to blob storage, and invoking Azure OpenAI to parse the combined metrics. By placing this coordination in a dedicated service, we kept the API controllers small. Other services include `ComplaintService`, which validates farmer complaints and routes them to regional officer queues based on district IDs, and `AuthenticationService`, which handles BCrypt password validation and issues JWT tokens.

6.5.2 DTOs

Passing raw Entity Framework models directly to the client creates two issues: it exposes our internal database schema, and it causes serialization cycles due to navigation properties (for instance, a User referencing a Complaint that references a User). To prevent this, we mapped database entities to Data Transfer Objects (DTOs) using AutoMapper. For example, `VerificationResponse` only returns the visual classification label, confidence score, and extracted DUS metrics, stripping out internal SQL primary keys and auditing properties. Using DTOs also reduced JSON payload sizes, which is important for mobile users operating on high-latency rural connections. The primary DTOs utilized in the system and their respective purposes are listed in Table 6.3.

Table 6.3: Important DTOs

DTO	**Purpose**
VerificationRequest	Seed verification request
VerificationResponse	AI verification result
LoginRequest	User login request
AuthResponse	JWT authentication response
CreateComplaintRequest	Complaint submission
OfficerDto	Officer information
AdminDashboardResponse	Dashboard statistics

6.5.3 Validation

To prevent malformed data from hitting the deep learning container or database, we write validation logic using FluentValidation. We intercept incoming requests before they reach the main service logic. The validator checks that uploaded files

have a valid MIME type (restricting uploads to JPEG or PNG), verifies that the payload does not exceed the 10MB server limit (though client-side compression usually keeps it under 300KB), and enforces password and email syntax constraints. This preemptive validation prevents downstream errors in the PyTorch model service and saves database connections.

6.6 Infrastructure Layer

The Infrastructure Layer implements the interfaces defined in our Domain layer, handling direct database connections and network communication with external APIs.

6.6.1 Database Management and Repositories

Database communication is configured using EF Core. While ORMs simplify basic database updates, they can introduce performance bottlenecks if queries are not carefully constructed. For example, to avoid the N+1 query problem—where the application executes a separate SQL query to load the user profile for every verification record displayed on the officer dashboard—we wrote optimized LINQ queries using `.Include()` statements. This forces EF Core to generate single SQL JOIN queries, returning all required data in one database trip. We also implemented the repository pattern to encapsulate these data access details behind the domain interfaces, keeping database-specific LINQ logic out of the application services. We use EF code-first migrations to apply schema updates incrementally, keeping development and production databases synchronized.

6.6.2 AI and Cloud Integrations

Our infrastructure implementations handle connection lifecycles for external cloud APIs. The external service integrations are detailed in Table 6.4.

Table 6.4: External Service Integrations

Service	**Purpose**
Azure OCR	Text extraction from seed packets
Azure OpenAI	AI-generated recommendations
Azure Blob Storage	Cloud image storage
ResNet Model	Paddy seed Quality Determination

When the verification service receives a compressed image, the storage adapter

uploads it to Azure Blob Storage and generates a read-only URL. This URL is dispatched to the Azure AI Vision OCR service to extract text from the packaging label, such as batch numbers and expiry dates. At the same time, the image is sent to our FastAPI service hosting the ResNet50 model. The results are aggregated, and the backend sends the combined data to the Azure OpenAI service, instructing the model to synthesize the raw statistics and OCR text into a plain-language summary for Meitei farmers.

6.7 Database Schema

The backend database maintains relationships between users, verification results, complaints, and audit records. The database schema and its corresponding primary/foreign key mappings are represented in the Entity-Relationship Diagram in Figure 6.2.

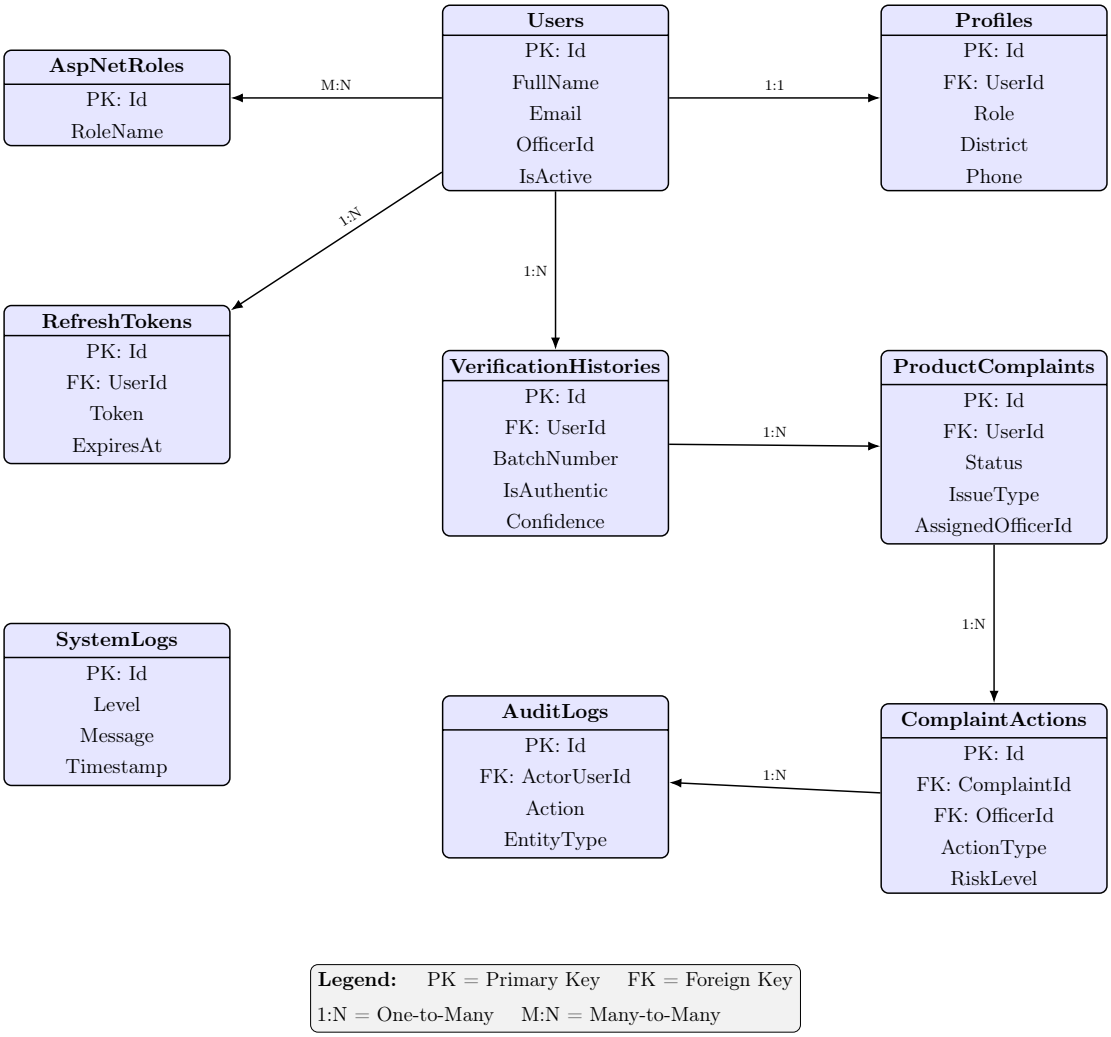

Figure 6.2: Entity Relationship Diagram – AgriVerify System

The database schema is structured around a relational model to maintain strong referential integrity. User accounts are linked in a one-to-one relationship with profiles containing role and region details. Verification records are tied directly to the submitting user and can be referenced by subsequent agricultural complaints if anomalies are detected. Complaints transition through multiple stages, with each action logged in a tracking table to maintain an audit trail. This prevents orphan records and ensures that all historical data remains traceable.

6.8 Presentation Layer

The Presentation Layer exposes the HTTP endpoints used by the Next.js frontend client.

6.8.1 API Controllers

The API controllers handle routing and request parsing. They extract the client's payload, run the FluentValidation middleware, and pass the data to the application services. Upon completion, the controllers map the internal result models to appropriate HTTP status codes, such as returning a 201 Created status for a successfully registered complaint, or a 400 Bad Request if the input image is blurry. This prevents raw internal exceptions from bubbling up to the client. The controllers and their routing purposes are summarized in Table 6.5.

Table 6.5: API Controllers

Controller	**Purpose**
AuthController	Authentication operations
VerificationController	Seed verification endpoints
ComplaintsController	Complaint management
AdminController	Administrative operations
AnalyticsController	Dashboard analytics
ReferenceDataController	Dropdown reference data

6.8.2 Middleware

We wrote custom middleware components to process HTTP requests before they hit the controllers. The authentication middleware extracts the JWT from the request header, decodes the signature using our symmetric key, and populates the current user context. The logging middleware uses Serilog to record the execution time of each request, which helped us locate performance bottlenecks

in our downstream OCR integrations. Finally, the exception-handling middleware captures uncaught runtime errors. Instead of displaying default stack traces that expose database table names and directory paths, the middleware logs the error details internally and returns a structured JSON error response to the client.

6.9 Authentication and Security

Authentication and authorization are managed through JSON Web Tokens (JWT) to support a stateless API architecture. The validation pipeline is illustrated in Figure 6.3.

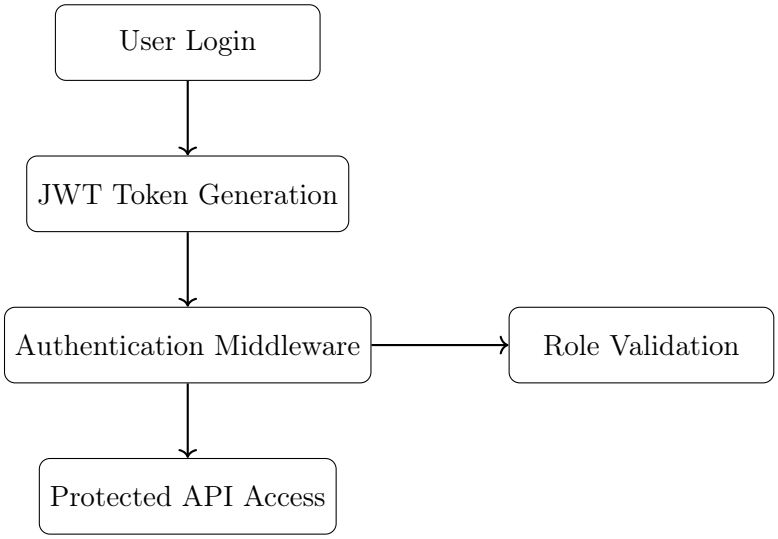

Figure 6.3: JWT Authentication Flow

Authentication relies on JSON Web Tokens (JWT) to support a stateless API architecture. During authentication, the backend hashes incoming passwords using BCrypt and compares them against the stored hash. If valid, the system generates an access token with a 15-minute lifespan and a cryptographically signed refresh token stored in an HttpOnly cookie to mitigate cross-site scripting (XSS) risks. When the short-lived access token expires, the client proxy automatically submits the refresh token to obtain a new access token, preventing session disruption while minimizing the impact of a compromised access token. Role validation is performed during this middleware pass, blocking farmers from accessing officer endpoints.

6.10 Verification Workflow

The seed verification workflow coordinates image validation, deep learning classification, text extraction, and database persistence, as shown in Figure 6.4.

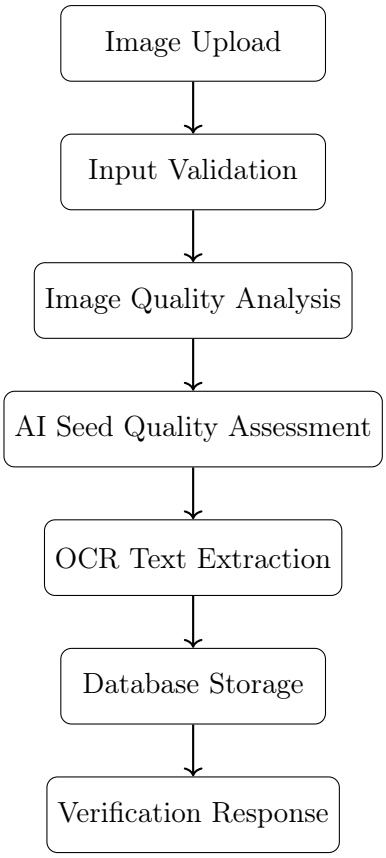

Figure 6.4: Seed Verification Workflow

The seed verification workflow handles image validation, AI inference, and database persistence. When a farmer submits an image, the API controller validates the upload headers and checks for basic image quality, such as assessing if the image is too blurry or dark, which would yield inaccurate model classifications. If the check passes, the image is written to Azure Blob Storage, and the system concurrently calls the ResNet50 FastAPI classification endpoint and the Azure OCR text extraction API. Once both responses return, the system saves the classification scores, morphological parameters, and label text to SQL Server. Finally, the backend feeds these inputs to Azure OpenAI to generate a clear advisory report, returnable as a unified JSON response.

6.11 Configuration and Deployment

We split application configurations into environment-specific files: `appsettings.json`, `appsettings.Development.json`, and `appsettings.Production.json`. During local development, the app connects to a local database instance and disables HTTPS enforcement. In production, we override these settings using environment variables injected by our container

hosting platform, securing our database credentials, JWT security keys, and Azure service passwords.

During startup, the `Program.cs` entry point reads these parameters to register database context classes, configure our dependency injection containers, enable JWT validation rules, and build the HTTP request pipeline. This allows us to run the application with minimal dependencies during local development and easily switch to cloud databases and monitoring in our production environment.

Chapter 7

Frontend System Architecture and Implementation

The AgriVerify frontend provides a responsive and secure browser interface that connects users to the backend services. It is designed to accommodate two distinct user profiles: farmers accessing the system via mid-range Android devices over unstable field networks, and state administrators managing comprehensive supply chain dashboards. The technology stack Next.js 16, React 19, TypeScript 5 and Tailwind CSS was selected to ensure optimal client-side performance, accessibility and clean architectural separation of concerns without sacrificing maintainability [24], [25].

7.1 Frontend Architecture and Design Patterns

The architecture separates UI rendering from data fetching, API communication and authorization. Each concern lives in its own layer.

7.1.1 Architectural Layers

A request passes through four layers before reaching backend services (Figure 7.1):

1. **Presentation UI (React & Next.js):** Handles component rendering, form validation and responsive layout [26].
2. **Client-Side State (TanStack Query):** Manages async data fetching, caching and optimistic mutations [27].

3. **Service Client (Axios):** Wraps network calls, standardising payloads and error handling.

4. **API Proxy (Next.js Routes):** Routes client requests to ASP.NET Core endpoints without exposing internal URLs to the browser.

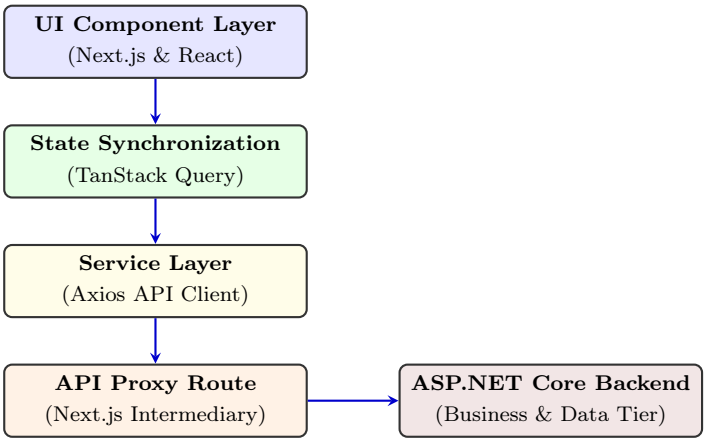

Figure 7.1: Frontend Layered Architecture and Request Execution Flow

7.1.2 Secure Proxy Architecture

All outbound requests route through internal Next.js serverless endpoints (`/api/proxy/*`), which avoids CORS issues by keeping cross-origin communication server-side. Internal microservice addresses stay out of the browser, secrets remain on the server, and security headers are applied before responses reach the client [28].

7.2 Technology Stack Analysis

Table 7.1 summarises the stack and the reasoning behind each choice.

Table 7.1: Frontend Technology Stack

Technology	**Architectural Reasoning**
Next.js 14	Provides Server-Side Rendering and built-in API proxy routing [29].
React 18	Declarative UI rendering with efficient Virtual DOM reconciliation.
TypeScript 5	Compile-time type checking with backend DTO parity.
Tailwind CSS	Utility-first styling with minimal production bundle output [13].
Axios & TanStack	HTTP pipeline management, token injection, and cache synchronization [27].
Zustand 4.5	Lightweight atomic state slices for session persistence.

7.3 Modular Project Structure

The source tree follows domain-driven modularity. Key directories: `src/app` (Next.js routing), `src/components` (reusable UI primitives), `src/hooks` (data fetching logic), and `src/types` (TypeScript DTOs matching backend entities). Keeping these separate means business logic and presentation concerns do not bleed into each other.

7.4 User Role-Based Access Control (RBAC)

The frontend uses Role-Based Access Control (RBAC) to separate public and protected routes.

7.4.1 Role Partitioning

The application supports four user profiles: **Farmers** (verification and dispute submission), **Agricultural Officers** (inspection queues and evidence review), **Administrators** (system governance and telemetry) and **Public Users** (anonymous checks).

7.4.2 Route Protection Middleware

A `RoleGuard` component checks JWT claims against required permissions before mounting any protected React component, preventing privilege escalation at the route level. The core implementation logic is detailed in Listing 7.1.

```tsx
export const RoleGuard: React.FC<{ children: React.ReactNode;
    allowedRoles: Role[] }> = ({ children, allowedRoles }) => {
    const { token, role, isLoading } = useAuthStore();
    const router = useRouter();

    useEffect(() => {
        if (!isLoading) {
            if (!token) return router.replace('/login');
            if (role && !allowedRoles.includes(role)) return router.
    replace('/unauthorized');
        }
    }, [token, role, isLoading, router, allowedRoles]);

    if (isLoading || !token) return <Loader />;
    return <>{children}</>;
};
```

Listing 7.1: Core Logic of the Role-Based Route Protection Guard

7.5 Authentication and Secure Persistence

Sessions are secured using a dual-token JWT exchange designed to limit exposure to XSS attacks [28].

7.5.1 Token Storage Architecture

Short-lived access tokens are held in the Zustand in-memory store, keeping them out of reach of malicious scripts. Long-lived refresh tokens are persisted in encrypted storage or `HttpOnly` cookies, allowing background session renewal without exposing token values to client-side code.

7.5.2 Automated Token Rotation

Axios interceptors handle token expiry without user intervention. On a `401 Unauthorized` response, the interceptor pauses pending requests, runs a refresh cycle, and replays the original request with the new token. From the user's side, the session continues uninterrupted. The automated refresh interceptor configuration is shown in Listing 7.2.

```tsx
apiClient.interceptors.response.use(res => res, async (error) => {
    const origReq = error.config;
    if (error.response?.status === 401 && !origReq._retry) {
```

```
        origReq._retry = true;
        try {
            const res = await axios.post('/api/proxy/auth/refresh', {
    token: useAuthStore.getState().refreshToken });
            useAuthStore.getState().setTokens(res.data.accessToken,
    res.data.refreshToken);
            origReq.headers['Authorization'] = `Bearer ${res.data.
    accessToken}`;
            return apiClient(origReq);
        } catch (err) {
            useAuthStore.getState().logout();
            return Promise.reject(err);
        }
    }
    return Promise.reject(error);
});
```

Listing 7.2: Axios Interceptor for Automated JWT Token Rotation

7.6 API Integration and Type Safety

TypeScript DTOs enforce compile-time verification of HTTP payloads across all integration modules Auth, Verification, Analytics, and Complaint Services. Each mirrors the corresponding ASP.NET Core schema, so data arriving at the frontend matches what the UI expects without runtime coercion or defensive casting.

7.7 State Management

State is split by volatility: server-driven data and client-local data are managed separately.

7.7.1 Asynchronous Server State (TanStack Query)

Network-driven data (dispute queues, telemetry) is managed by TanStack Query, which handles stale-while-revalidate caching, background polling, and optimistic UI updates. On slow connections, this keeps the interface usable even when the server is slow to respond [27].

7.7.2 Synchronous Client State (Zustand)

UI state that doesn't touch the network (active theme, session context) lives in Zustand. Its atomic slices limit unnecessary re-renders, which matters on lower-end devices where battery and memory are constrained.

7.8 Responsive UI Design and Field Usability

The UI is built mobile-first using Tailwind CSS. Most farmers will access the application on a mid-range Android handset, often outdoors, so the interface is designed around that constraint rather than treating it as an edge case [25].

7.8.1 Standardized UI Library

The interface is built from a set of reusable primitives (Table 7.2) to keep visual behaviour consistent across screens.

Table 7.2: Standardized UI Components

Component	**Role**
Data Table	Scrollable grids with sticky headers and dynamic pagination for inspection data.
Status Badge	Colour-coded indicators (e.g., Green/Genuine) for fast visual parsing.
Loading Skeleton	Animated placeholders that prevent layout shifts during network delays.
Input Modal	Focused overlays that trap keyboard navigation for accessible data entry.

7.8.2 Accessibility Enhancements

The UI meets WCAG AAA contrast ratios for readability in bright outdoor conditions, uses a minimum touch target size of 48×48px to reduce input errors, and includes full ARIA semantic tagging for screen-reader compatibility.

Chapter 8

System Integration

This chapter details how the Next.js frontend client, the ASP.NET Core backend, and the FastAPI model serving service communicate. Running these components as independent services allows us to scale them separately based on request volume. For example, the visual frontend and deep learning microservice can scale dynamically during peak harvesting periods without requiring continuous database server upgrades.

8.1 System Integration

Communication in the AgriVerify platform traverses distinct host boundaries. Initially, hosting the Next.js client on Vercel and the ASP.NET Core API on Render triggered browser-level CORS errors and SameSite cookie blocking. To resolve this, we configured Next.js server-side API proxy routes. The client browser makes requests directly to the Next.js origin, which forwards them to the Render backend server, handling JWT token injections server-side. This layout bypasses browser same-site restrictions and prevents public exposure of our backend API endpoints. Table 8.1 summarizes our service integration paths and security implementations.

8.2 End-to-End Architectural Workflow

Handling image files from modern mobile devices presented a significant design challenge. High-resolution cameras on mid-range smartphones produce large JPEG images (often between 15MB and 20MB). Uploading these raw files over unstable 3G and 4G networks in rural districts like Senapati and Tamenglong frequently resulted in HTTP request timeouts.

Table 8.1: System Integration and Communication Matrix

Communication Path	**Protocol**	**Technical Implementation & Security**
Browser to Proxy	HTTPS with multipart uploads	Client-side Axios interceptors automatically inject JSON Web Tokens (JWT) into request headers and coordinate silent token rotation upon expiry [30], [31].
Proxy to Backend	REST API over HTTPS	The proxy gateway intercepts incoming requests, performs schema validation and forwards payloads to internal ASP.NET Core endpoints, encapsulating internal network topology [32].
Backend to AI	REST (HTTPS)	The backend orchestrates parallel requests, transmitting image binaries to dedicated AI services for quality checks (blur/illumination detection) and Quality Assessment models (ResNet50) [4].
Cloud Services	Azure SDK Client Libraries	Azure OpenAI Service for text synthesis and Azure Blob Storage for unstructured binary object persistence [33].
Deployment	Container Orchestration	Multi-stage Docker builds compile and optimize service codebases, which are stored in the Azure Container Registry and deployed using container orchestration tools [20].

To make the app usable in areas with weak cellular signals, we integrated client-side compression in the browser using the `browser-image-compression` library. The script downscales images to a maximum width of 1024 pixels before upload, reducing the file size to under 300KB while preserving enough visual detail for the classification model. This client-side optimization resolved both client upload timeouts and the payload size limit in our Next.js proxy route.

The lifecycle of a verification request spans five processing stages, illustrated in Figure 8.1.

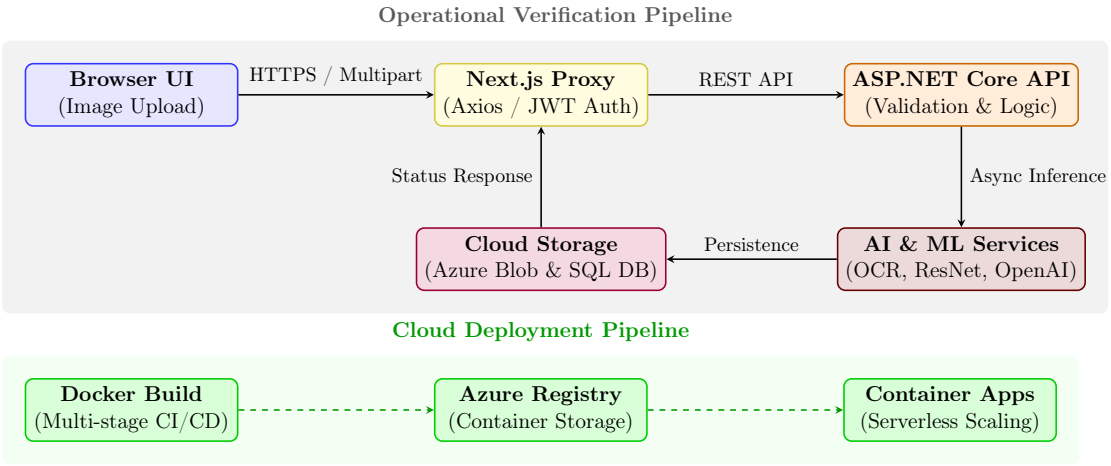

Figure 8.1: Architectural Workflow and Cloud Deployment Pipeline

The request execution pipeline follows a linear path:

1. **Client-Side Compression and Routing:** When a user uploads a photo, the browser-based JavaScript downsamples the image. The client pushes this payload to our Next.js API proxy, which appends the JWT authorization token to the request headers, automatically requesting a token refresh if the signature has expired [30], [31].

2. **API Ingestion and Validation:** The Next.js server forwards the request to the ASP.NET Core backend. The API controllers run FluentValidation rules to ensure the multipart upload headers and file sizes are correct before triggering the AI workflow [32].

3. **Parallel AI Processing:** The backend coordinates downstream microservices. It forwards the image to the FastAPI model service for ResNet50 classification while simultaneously calling the Azure Vision OCR API to read the seed packet label. To resolve startup latency on GCP Cloud Run serverless instances, we configured the FastAPI service to maintain a minimum of one warm instance during active agricultural periods, avoiding a 15-second cold-start delay.

4. **Blob Storage and SQL Mapping:** After classification and text extraction complete, the backend uploads the seed image to Azure Blob Storage. Rather than storing binary image files directly in SQL Server, which would inflate database size and degrade query performance, we store only the image URLs alongside the classification scores and batch numbers [33].

5. **Report Synthesis and Rendering:** The backend sends the combined classification statistics and OCR text to Azure OpenAI, which compiles a summary for Meitei farmers. The backend returns this text as a JSON response to the Next.js client, which renders it on the user dashboard.

We automate service builds using GitHub Actions. Pushes to our main branch trigger a runner that compiles each service into a Docker image. To reduce build times and final image sizes, we use multi-stage Dockerfiles that compile the application code in a builder environment and copy only the final binaries to the runtime base. The runner pushes the optimized images to Azure Container Registry, triggering automatic deployment updates on Azure Container Apps [20].

8.2.1 Unexpected Integration Challenges and Resolutions

During system integration, we encountered several unexpected challenges related to cross-domain security and downstream service latency that required architectural modifications. The first major challenge arose when linking our Next.js frontend, hosted on Vercel, with the ASP.NET Core backend API, hosted on Render. Because these services reside on different domains, modern web browsers blocked authorization cookies due to strict SameSite cookie policies. To resolve this without compromising session security, we routed all client requests through a Next.js server-side API proxy. This server-side route acts as an intermediary, injecting JWT tokens and forwarding request payloads to the backend Render server, successfully bypassing CORS restrictions and cross-domain cookie blocks.

A second integration challenge concerned meeting our non-functional requirement of keeping the end-to-end inference latency under 3.0 seconds. Initially, the backend orchestrated calls to Azure Blob Storage, the ResNet50 FastAPI service, Azure OCR, and Azure OpenAI sequentially. This sequential execution caused cumulative latency to exceed 7.0 seconds per request, resulting in browser timeout errors. To optimize this pipeline, we refactored the ASP.NET Core orchestrator to process independent operations in parallel. By executing the CPU-bound ResNet50 classification and Azure OCR text extraction concurrently using C# asynchronous tasks (`Task.WhenAll`), we reduced the overall request-response window to under 2.5

seconds, ensuring a highly responsive user experience even on slower networks.

8.2.2 Live Prototype Deployment

For our live prototype demonstration and evaluation, the Next.js frontend is deployed on Vercel, and the ASP.NET Core backend is hosted on Render. These environments allow us to run real-world tests with active users. The live production endpoints can be accessed at the following URLs:

- Deployed Frontend Client: `https://agriverify02.vercel.app/`
- Deployed Backend API Gateway: `https://fakeseeddetection.onrender.com/`

The Next.js API proxy routes all client-side queries directly to the Render backend endpoint, which processes the request payloads and coordinates downstream AI inference tasks.

Chapter 9

Results and Evaluation

9.1 Model Performance

After completing the training phase, we evaluated the ResNet50 model using an independent test dataset consisting of 231 previously unseen images. We quantified model performance using standard statistical metrics: Accuracy, Precision, Recall, and the F1 Score [34]. These metrics provide a multi-dimensional assessment of the model's capacity to distinguish between genuine and defective (damaged or counterfeit) paddy seeds [4], [8]. During the training process, we observed that classification loss fell sharply during the first six epochs and stabilized after epoch ten. The validation accuracy closely tracked the training accuracy, displaying only a marginal deviation of 2.8% at convergence, which indicates that our regularization and data augmentation strategies successfully prevented overfitting.

The evaluation results demonstrate high model performance. The model consistently classified seed samples into two primary categories: `Pure` (certified, legitimate seeds) and `Impure` (damaged, defective, or counterfeit seeds). The empirical results suggest that the AgriVerify system is suitable for deployment in real-world agricultural scenarios, particularly within resource-constrained environments.

9.1.1 Accuracy, Precision, Recall, and F1 Score

The overall performance metrics of the model on the test dataset are summarized in Table 9.1. Furthermore, the convergence behavior of the model is analyzed across multiple metrics: the training and validation accuracy profile over 10 epochs is illustrated in Figure 9.1, and the corresponding training and validation loss curves are shown in Figure 9.2. Additionally, Figure 9.3 displays the precision, recall, and F1 score trajectories, while the training-to-validation generalization gap is tracked

in Figure 9.4.

Table 9.1: Overall Model Evaluation Results

Performance Metric	**Result**	**What This Means**
Accuracy	99.57%	Out of every 100 seeds we tested, we correctly classified 99 or 100 of them
Precision	99.57%	When the system says a seed is problematic, it's right almost every time—virtually no false alarms
Recall	99.57%	The system catches almost all problematic seeds—very few slip through undetected
F1 Score	99.57%	Perfect balance between precision and recall—the system is equally good at both

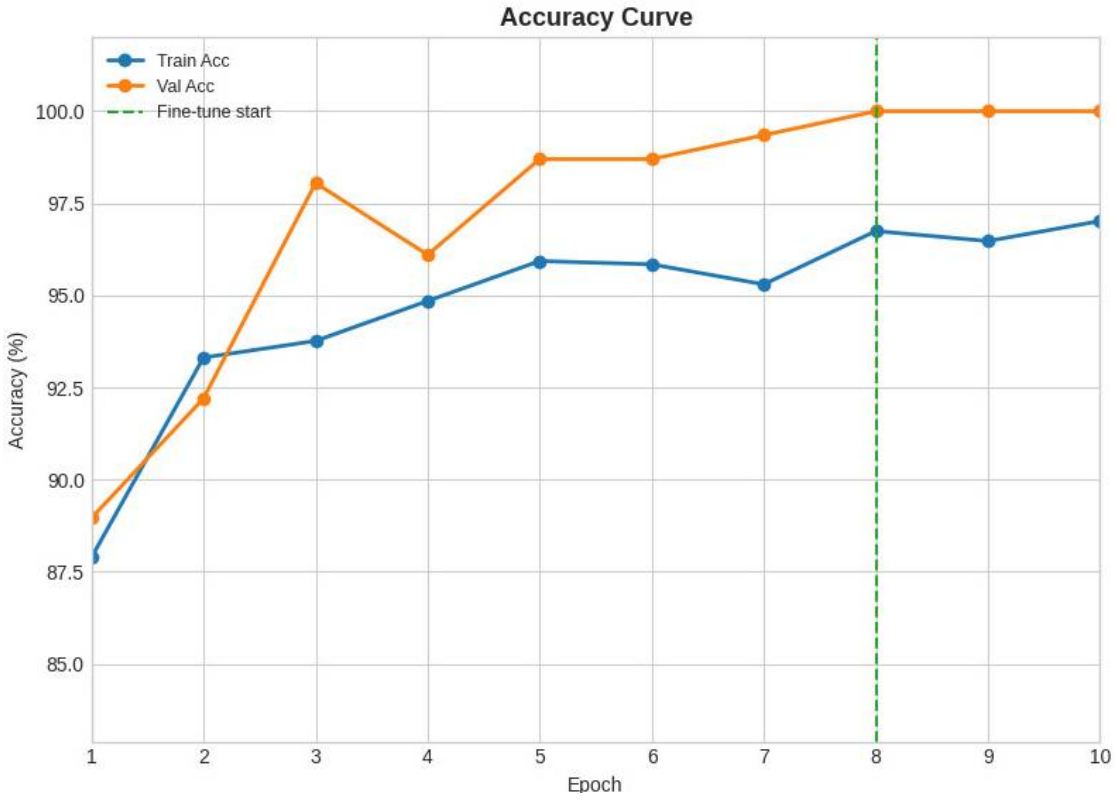

Figure 9.1: Training and validation accuracy curves across 10 epochs.

An accuracy rate of 99.57% indicates that the model successfully generalizes features across varying seed textures and illumination conditions. The equivalence of precision and recall (both at 99.57%) is critical for agricultural applications; it signifies a minimized rate of both false positives (misclassifying impure seeds as pure) and false negatives (misclassifying pure seeds as impure). The F1 Score confirms a robust balance between these two critical objectives [35].

As shown in the accuracy profile in Figure 9.1, the training accuracy begins at approximately 88.0% at the first epoch and climbs steadily to reach 97.0% by epoch

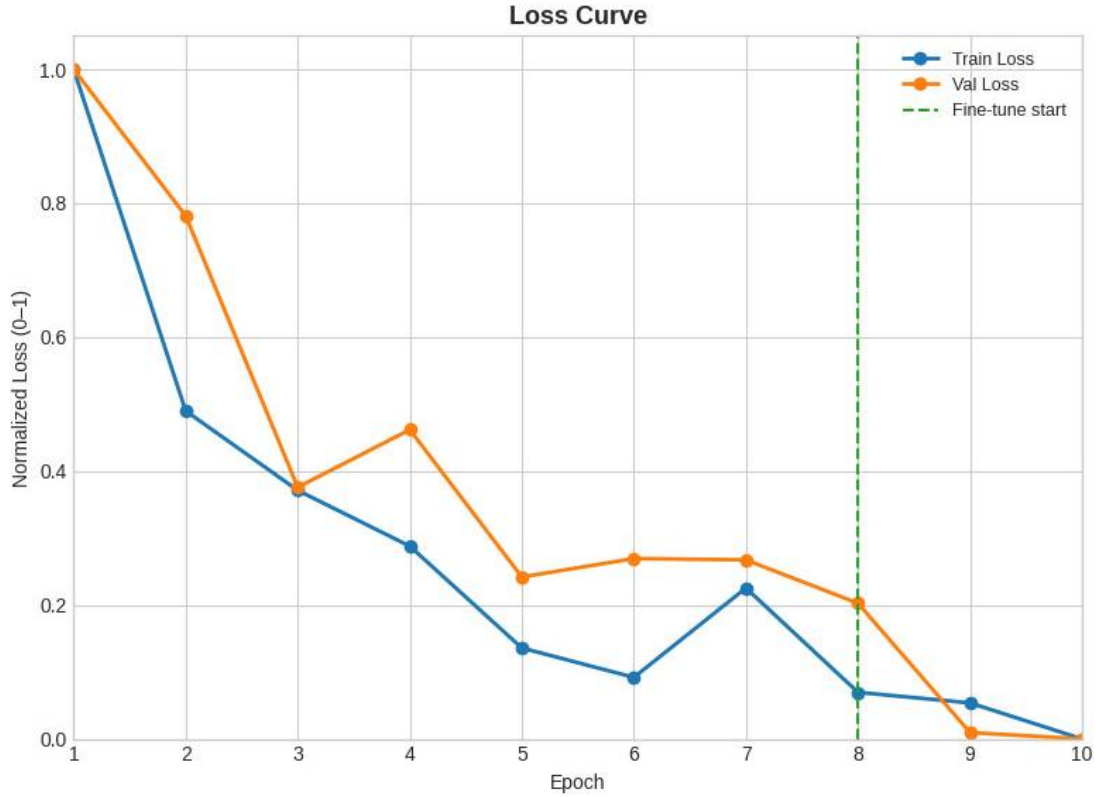

Figure 9.2: Training and validation normalized loss curves across 10 epochs.

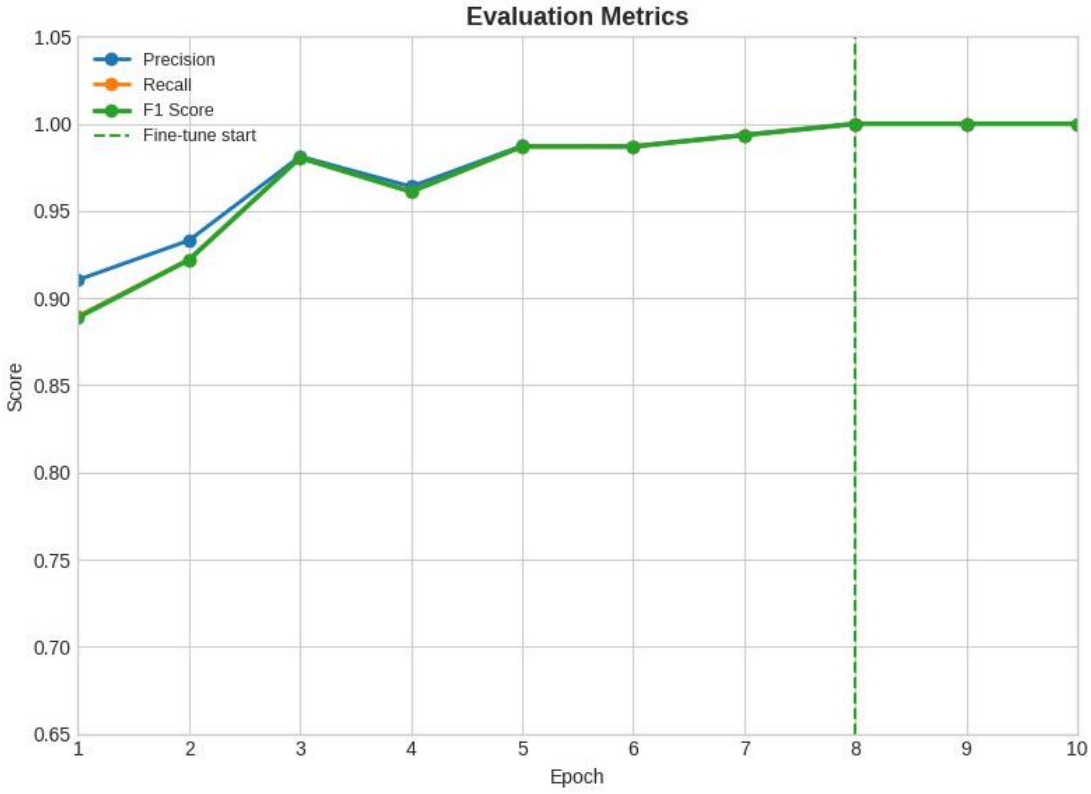

Figure 9.3: Precision, Recall, and F1 Score trajectories across 10 epochs.

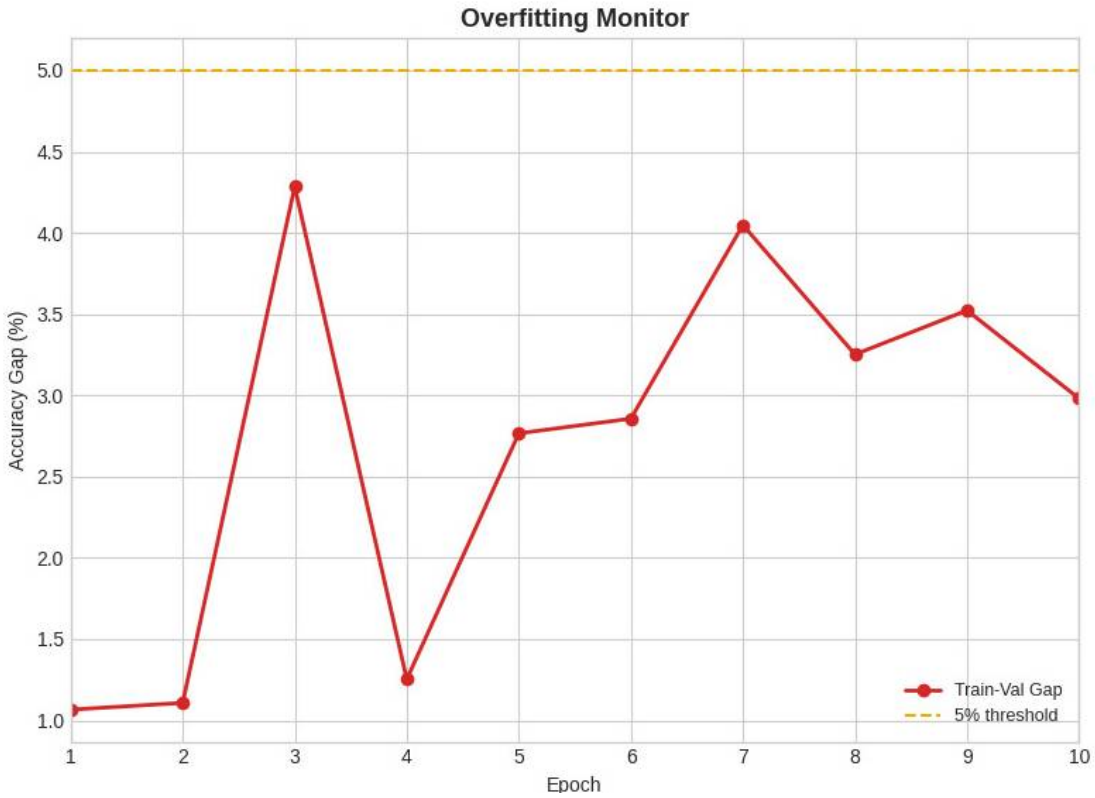

Figure 9.4: Generalization gap (Train-Val difference) monitored against a 5% overfitting threshold.

10. The validation accuracy starts slightly higher at 89.0% and exhibits a rapid ascent, reaching 100.0% at epoch 8, which corresponds to the activation of the fine-tuning phase (indicated by the dashed line). This high validation accuracy is maintained stably through to epoch 10. The normalized loss curves in Figure 9.2 exhibit a complementary trend. The training loss declines consistently from 1.0 down to approximately 0.0, while the validation loss falls rapidly from 1.0 to 0.37 at epoch 3. A transient fluctuation is observed at epoch 4 (increasing to 0.46) before the loss continues its downward trajectory, converging closely to 0.0 following the start of fine-tuning at epoch 8.

The model's primary classification metrics are tracked in Figure 9.3. The precision, recall, and F1 score start at 91.0%, 89.0%, and 89.0% respectively, and demonstrate steady improvement. At epoch 8, all three curves converge at 1.0 (100.0%) and remain stable, illustrating the positive impact of the fine-tuning phase on the model's discriminative capability. Furthermore, the generalization behavior is tracked in Figure 9.4, which displays the percentage gap between training and validation accuracy. The generalization gap starts at 1.1%, experiences a peak of 4.3% at epoch 3, and subsequently fluctuates before converging to 3.0% at epoch 10. Crucially, the gap remains entirely below the designated 5% overfitting threshold

(marked by the horizontal dashed line), verifying that the model generalizes robustly to unseen data without memorizing noise.

From the accuracy curve (Figure 9.1), notably the accuracy of 99.57% surpasses the typical baseline of 85–90% reported in existing literature for automated seed quality assessment systems [2], [6]. This performance gain validates the integration of transfer learning with tailored image augmentation and regularization techniques.

9.2 Seed Quality Results: Pure vs. Impure Seeds

To evaluate the class-specific performance of the classifier, metrics were computed independently for the pure and impure seed categories. The test set comprised 231 samples distributed across both classes, with the results summarized in Table 9.2.

Table 9.2: Performance on Each Seed Category

Seed Type	**Precision**	**Recall**	**F1 Score**	**What This Means**
Genuine	100.0%	99.19%	99.60%	Perfect at recognizing genuine seeds never wrongly approves a low quality one
Low Quality	99.07%	100.0%	99.53%	Perfect at detecting problems catches every single defective seed

As shown in Table 9.2, the model achieved 100.0% precision for the pure category, which can be attributed to the highly distinct visual markers of genuine paddy seeds, such as uniform golden-brown coloration, intact husks, and consistent surface textures. In contrast, the model demonstrated 100.0% recall for the impure category, successfully identifying all damaged, defective, or counterfeit seeds. This asymmetric performance is highly desirable for agricultural quality control, ensuring that sub-standard seed batches are consistently intercepted while minimizing the false rejection of certified stock [7].

The high performance is mainly due to the class-weighted loss functions and data augmentation strategies employed during training. These techniques mitigate class imbalance issues and prevent the network from exhibiting bias toward the majority class.

9.3 Confusion Matrix Analysis

To analyze the distribution of errors, a confusion matrix was constructed. The quantitative prediction distributions are detailed in Table 9.3, and the corresponding normalized confusion matrix is visualized in Figure 9.5.

Table 9.3: Detailed Look at Every Prediction

Actual Seed Type	**What the Model Predicted**	
	Impure	**Pure**
Impure	107	0
Pure	1	123

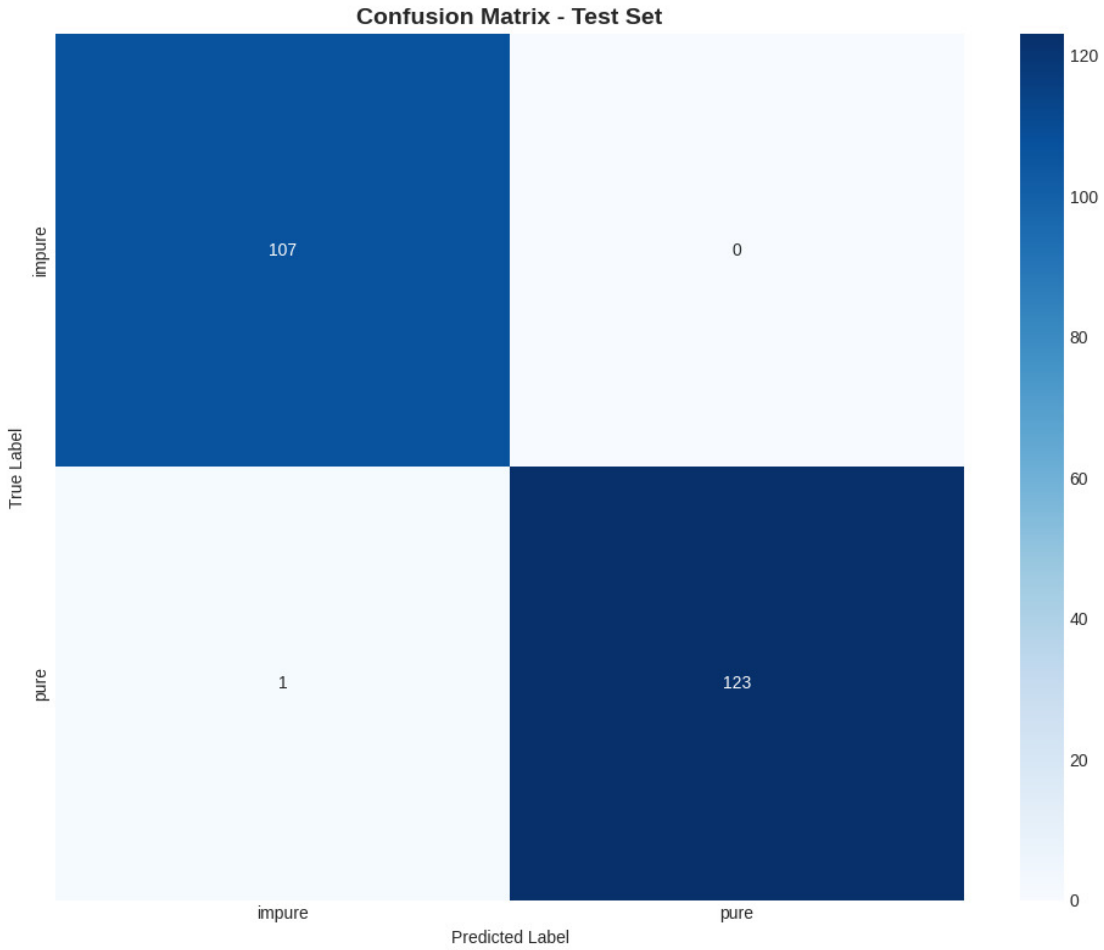

Figure 9.5: Visual representation of Confusion Matrix

From Table 9.3 and Figure 9.5, we observed that out of the 231 test samples evaluated, the model correctly classified 230 samples. The specific model outcomes are analyzed below:

Impure Class (107 Samples): All 107 impure samples were correctly classified as impure (True Positives), yielding zero false negatives. From an agricultural safety

perspective, this prevents the inadvertent distribution and planting of defective or counterfeit seed batches.

Pure Class (124 Samples): The model correctly identified 123 pure samples (True Negatives), with only 1 sample misclassified as impure (False Positive). This conservative error is preferable in agricultural workflows; while a false positive merely necessitates a secondary manual inspection, a false negative (failing to identify impure seeds) could lead to complete crop failure and economic losses for the farmer.

9.4 Regularization Effectiveness

With a dataset size of 1,563 images, training deep neural networks introduces a significant risk of overfitting, where the model memorizes high-frequency noise in the training set rather than extracting generalizable morphological features. To mitigate this, several regularization techniques were evaluated, as compared in Table 9.4.

Table 9.4: How Our Anti-Overfitting Techniques Helped

Configuration	**Train vs. Validation Gap**	**What Happened**
System with overfitting	25–30%	Bad—the model memorized training data and performed worse on unseen seeds
Dropout Only	8–12%	Better, but still some overfitting on tricky seed textures
All Techniques Combined	2.8%	Excellent—tight control with less than 5% difference

From Table 9.4, we observed that the validation-to-training accuracy gap was reduced to 2.8% when all regularization methods were active. This small generalization error indicates that the network successfully extracted generalizable features. This was achieved through the following architectural and optimization strategies:

- **Weight Decay (AdamW):** L2 regularization was applied to penalize large weight magnitudes, thereby constraining model complexity and improving generalization [36].
- **Label Smoothing:** Soft targets were introduced during cross-entropy loss computation to prevent the model from outputting overconfident logit predictions, which enhances calibration [37].

- **Dropout and Batch Normalization:** A dropout rate of 0.5 was applied to the fully connected layers to prevent co-adaptation of features, while batch normalization was utilized to stabilize internal covariate shift [4].
- **Learning Rate Scheduling:** A cosine annealing scheduler was implemented to dynamically decay the learning rate, allowing the optimizer to navigate complex loss landscapes and converge to flatter local minima.
- **Data Augmentation:** Diverse geometric and photometric transformations—including rotation, flipping, Gaussian blurring, and color jittering—were applied to simulate environmental variations encountered during field photography.

9.5 Model Comparison and Architecture Selection

Prior to selecting ResNet50 as the primary backbone, a comparative analysis was conducted using four prominent convolutional neural network architectures trained under identical hyperparameters and optimization settings. The quantitative evaluation metrics are compiled in Table 9.5. The comparative validation accuracy profiles across all evaluated backbones are illustrated in Figure 9.6, while the corresponding validation loss trajectories are presented in Figure 9.7.

Table 9.5: Comparing Four Different Neural Network Architectures

Model	**Accuracy**	**Precision**	**Recall**	**F1 Score**	**Training Time**
ResNet50	**99.57%**	**99.57%**	**99.57%**	**99.57%**	16.4 min
DenseNet121	99.57%	99.57%	99.57%	**99.57%**	15.0 min
GoogleNet	99.57%	99.57%	99.57%	**99.57%**	12.2 min
MobileNetV2	99.13%	99.15%	99.13%	99.13%	**8.5 min**

As shown in Table 9.5, ResNet50, DenseNet121, and GoogLeNet achieved identical model accuracies of 99.57%. MobileNetV2 registered a slightly lower accuracy of 99.13% but required significantly less training time. ResNet50 was selected as the final architecture due to several specific advantages.

The detailed validation trajectories in Figure 9.6 and Figure 9.7 provide further insight into the training dynamics. In Figure 9.6, all four model architectures (MobileNetV2, DenseNet121, GoogLeNet, and ResNet50) initiate training with validation accuracy between 89.0% and 100.0%. Upon starting the fine-tuning phase at epoch 8 (marked by the vertical dashed line), ResNet50 exhibits a temporary transient dip to approximately 93.0%, which represents a typical reorganization

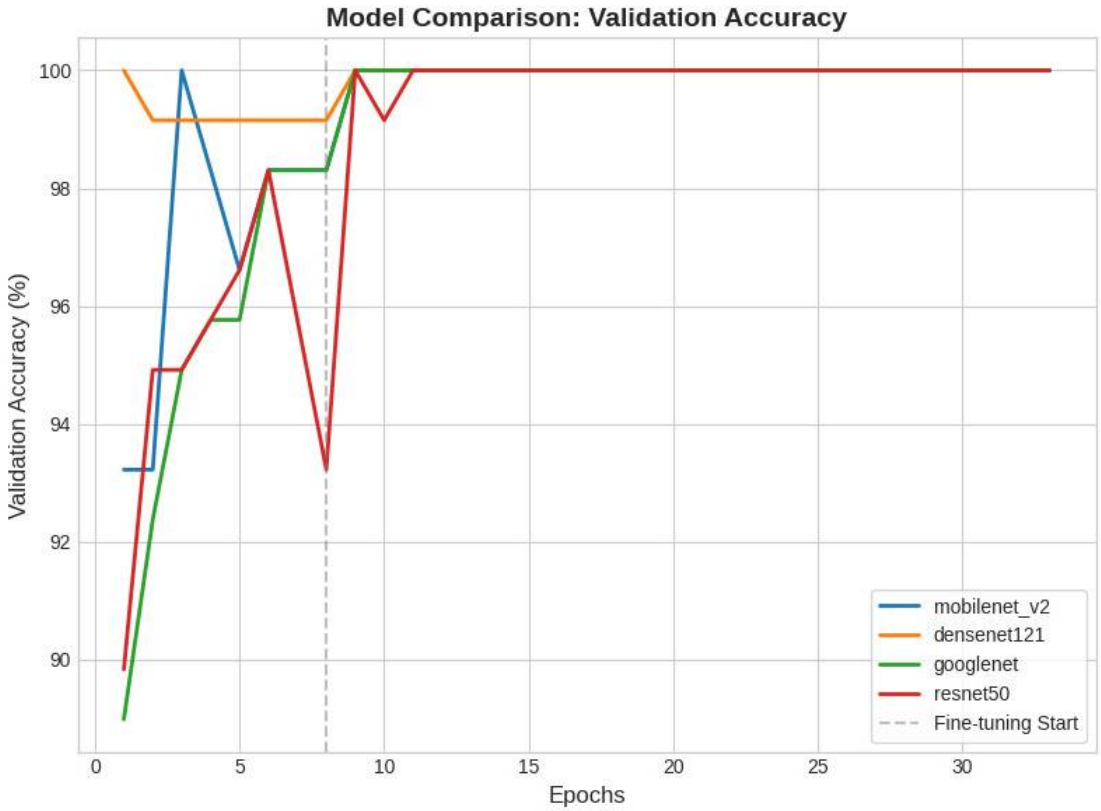

Figure 9.6: Comparison of validation accuracy profiles across different model architectures.

of weights in the deeper layers before rapid convergence. Following epoch 8, all architectures converge quickly to a stable validation accuracy of 100.0% and maintain this level through to epoch 30.

A corresponding convergence behavior is observed in the validation loss comparison in Figure 9.7. All models start with validation losses between 0.35 and 0.45. Prior to fine-tuning, the loss profiles show minor fluctuations, including a peak in ResNet50's validation loss to 0.39 at epoch 8. Following the fine-tuning start, the validation losses for all models decrease rapidly and settle at a minimum level around 0.325. Notably, GoogLeNet, ResNet50, and DenseNet121 achieve a slightly lower converged validation loss than the lightweight MobileNetV2, reflecting the benefit of their larger capacity and deeper feature representation capabilities.

The residual connections (skip connections) in ResNet50 address the vanishing gradient problem, allowing efficient backpropagation through the 50-layer architecture and facilitating fine-tuning without information loss [4], [8]. Furthermore, ResNet50 exhibits superior compatibility across deployment frameworks, including PyTorch, TorchScript, ONNX, and edge-optimized runtime engines. This cross-platform compatibility supports future plans to deploy AgriVerify as a native mobile application. The marginal training time overhead (an additional 4.4 minutes compared

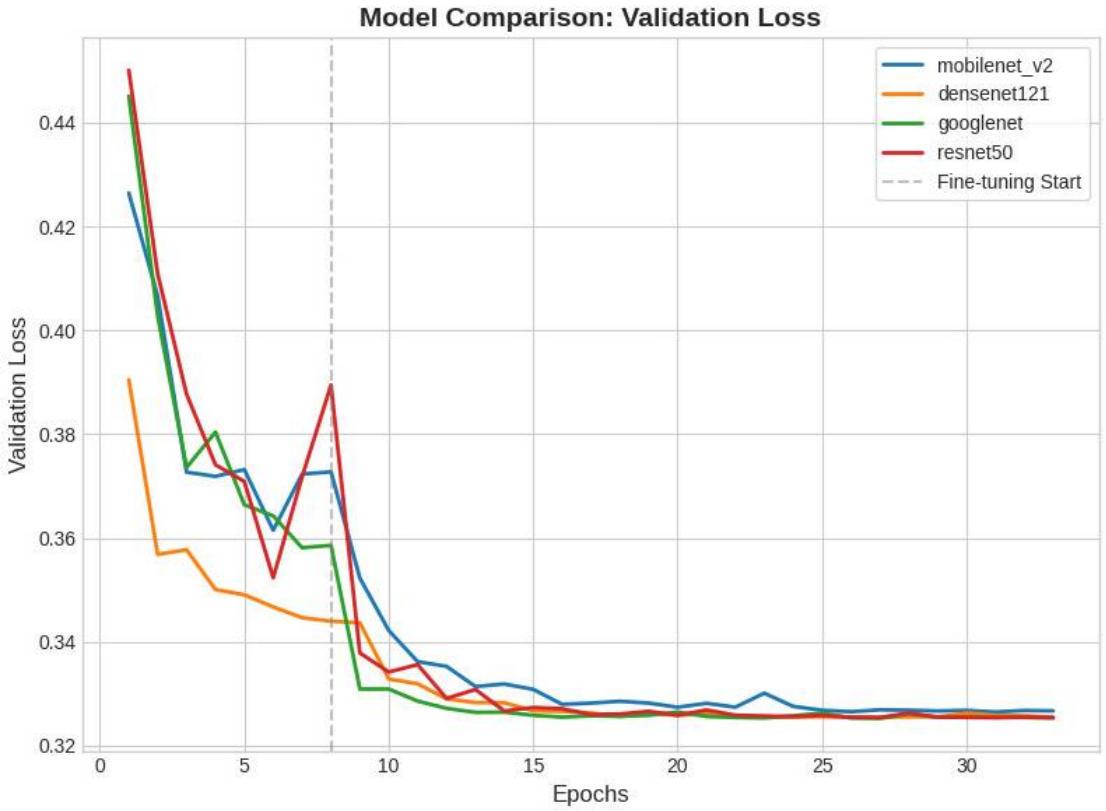

Figure 9.7: Comparison of validation loss profiles across different model architectures.

to GoogLeNet) is offset by these deployment advantages, given that model training is an offline process [15].

9.6 System Performance and Production Latency

In addition to model accuracy, computational efficiency and response latency are critical parameters for real-time applications deployed in agricultural fields.

9.6.1 Inference Latency Evaluation

Model inference latency refers to the duration required for the network to ingest a preprocessed image, perform a forward pass, and output the class probabilities. The latency profiles across different hardware configurations are summarized in Table 9.6.

On standard CPU architectures (without hardware acceleration), the model achieved an average inference latency of approximately 165 ms. When evaluated on GPU hardware (NVIDIA CUDA), the latency was reduced to 20 ms. Both

Table 9.6: Speed on Different Hardware

Hardware	**Speed**	**Good For What?**
Regular CPU	∼165 ms	Lightweight servers and edge devices; still plenty fast
GPU (NVIDIA CUDA)	∼20 ms	Cloud servers handling many requests; 8 times faster than CPU

execution speeds are well within acceptable thresholds for interactive, real-time application feedback.

9.6.2 End-to-End System Performance Under Load

To evaluate the scalability of the backend system under high concurrent access conditions, stress testing was conducted. The system throughput and latency metrics under simulated concurrent loads are detailed in Table 9.7.

Table 9.7: Real-World System Performance Under Load

Metric	**Observation**	**Significance**
End-to-End Response	280–420 ms	Includes uploading the image, sending it through the system, running AI, and formatting the response
Concurrent Requests	40–60 per second	Very stable even when many farmers submit seeds simultaneously
Image Upload & Prep	80–120 ms	Network transfer and image preprocessing before AI runs
Database Queries	<50 ms	Fast retrieval of complaint and user records

The system demonstrated high stability and responsiveness under peak concurrent loads. The asynchronous proxy design pattern prevents thread blocking at the core application layer by managing intensive operations outside the main request-response cycle, thereby preserving database and API availability [9].

9.7 Functional Verification and Test Execution

System validation was conducted to verify functional correctness, security enforcement and data integrity across all primary operational workflows.

9.7.1 Seed Quality Assessment Validation

The image processing pipeline and quality assessment subsystem were validated against diverse input scenarios. The specific test cases, expected behaviors, and execution results are detailed in Table 9.8.

Table 9.8: Seed Verification Test Cases

Test ID	What We Tested	Expected Behavior	Result
SV-01	Upload clear photos of genuine seed packaging	Correctly classified as Pure with physical measurements extracted	**Pass**
SV-02	Upload photos of cracked or discolored seeds	Classified as Impure (Damaged) with warnings shown	**Pass**
SV-04	Try uploading wrong file types (.exe, .zip, .txt)	System rejects and shows helpful error message	**Pass**
SV-05	Upload blurry or very dark seed photos	System warns user to retake photo or shows low confidence	**Pass**

9.7.2 Grievance Management Subsystem Validation

The complaint lodging and tracking functionalities were validated to verify database consistency and transactional security. The validation test cases are outlined in Table 9.9.

Table 9.9: Complaint System Test Cases

Test ID	What We Tested	Expected Behavior	Result
CM-01	Farmer files a complaint with description and evidence	Complaint saved to database and marked as Open	**Pass**
CM-02	Try filing complaint with missing required information	Form validation prevents submission and shows error	**Pass**
CM-03	Farmer views their complaint history	All past and current complaints displayed with pagination	**Pass**
CM-04	Officer changes complaint status from Open to Resolved	Update immediately visible in the system	**Pass**

9.7.3 Role-Based Access Control and Security Verification

To ensure secure operations, the authorization and authentication mechanisms were tested against access violations. The results of the security test cases are compiled in Table 9.10.

Table 9.10: Security and Access Control Test Cases

Test ID	What We Tested	Expected Behavior	Result
RB-01	Valid farmer login	Receives security token and goes to farmer dashboard	**Pass**
RB-02	Farmer tries accessing admin page	System blocks access and redirects to farmer area	**Pass**
RB-03	Admin login with multi-factor authentication	Full admin privileges granted	**Pass**
RB-04	Security token expires during session	System automatically gets new token without user noticing	**Pass**
RB-05	Login with wrong password	System returns error and shows helpful message	**Pass**

9.8 System User Interfaces and Operational Workflows

AgriVerify provides simple, intuitive interfaces tailored to three different user types: farmers, agricultural officers, and system administrators. We built these using modern web technologies: Next.js 15, React 19, and Tailwind CSS [13], [29], [38]. Let's walk through each interface and see how users interact with the system.

9.8.1 Public Portal, Onboarding, and Multilingual Support

Upon accessing the AgriVerify platform, users are presented with the public landing page, which outlines the system's value proposition and operational features (Figure 9.8). To accommodate rural demographics, the platform incorporates a localized language selection interface, enabling users to transition between English and regional dialects (Figure 9.9).

New users can register by selecting their role (Figure 9.10) and completing the onboarding registration form, which captures demographic, location, and agricultural district details (Figure 9.11). This location data is critical for geo-tagging and regional analytics in the grievance management system.

For administrative personnel and agricultural officers, a secure portal interface enforces multi-factor authentication (MFA) via time-based security codes to prevent unauthorized administrative actions (Figure 9.12).

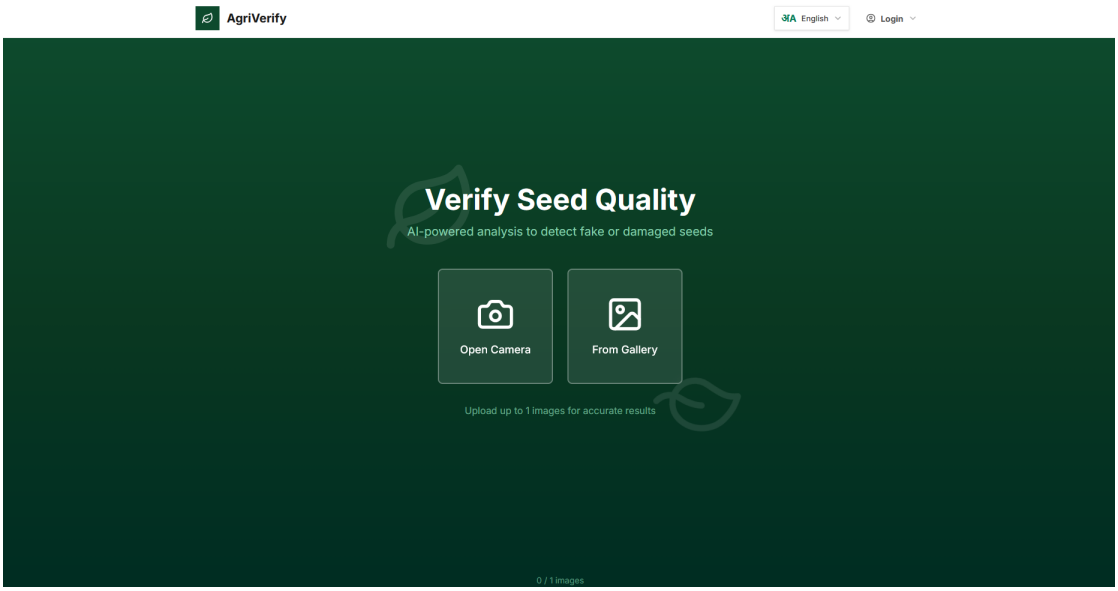

Figure 9.8: Platform landing page displaying value proposition and core features.

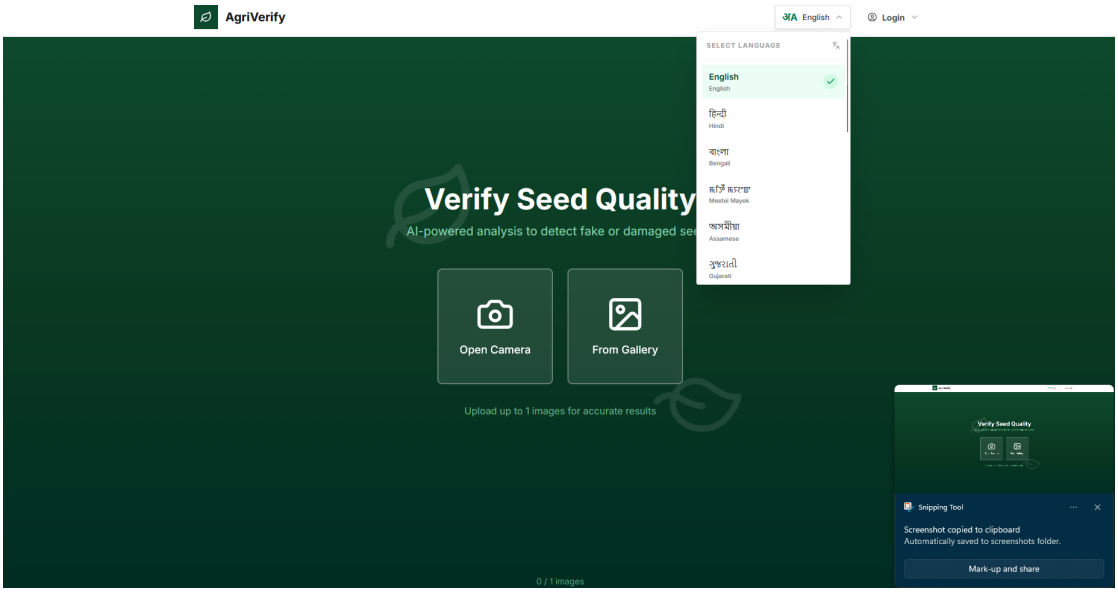

Figure 9.9: Multilingual selection interface supporting regional languages.

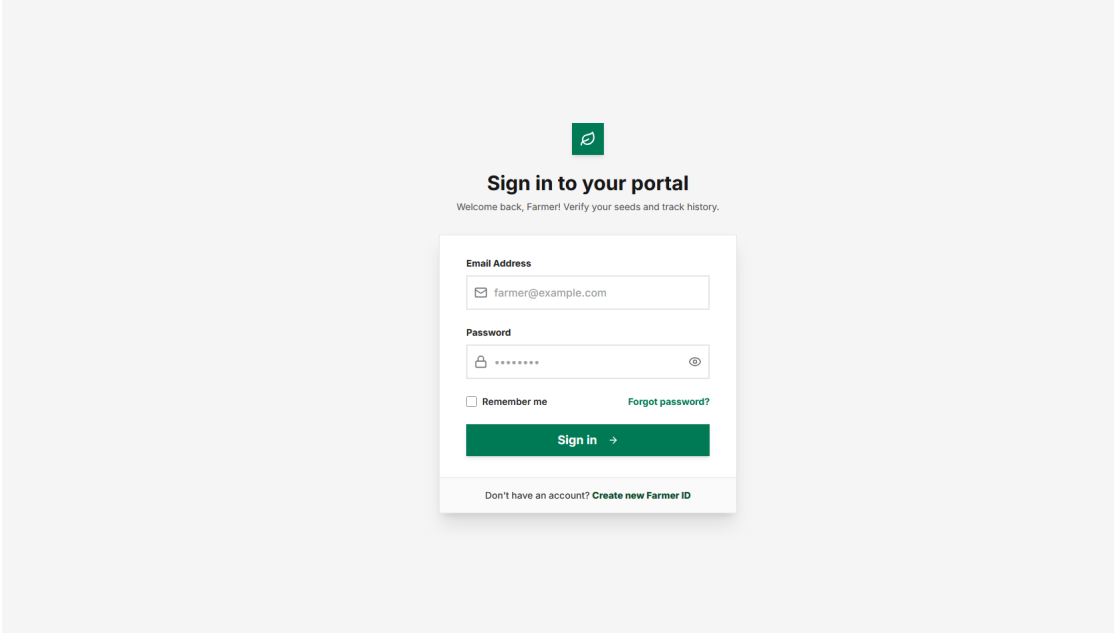

Figure 9.10: Role selection interface for login.

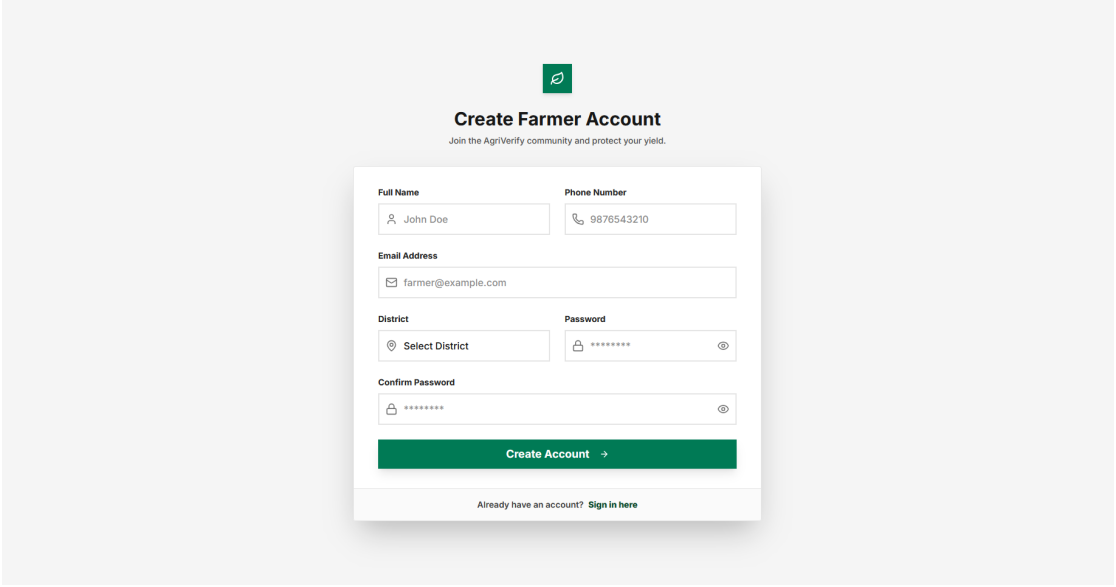

Figure 9.11: Farmer registration form collecting demographic and location information.

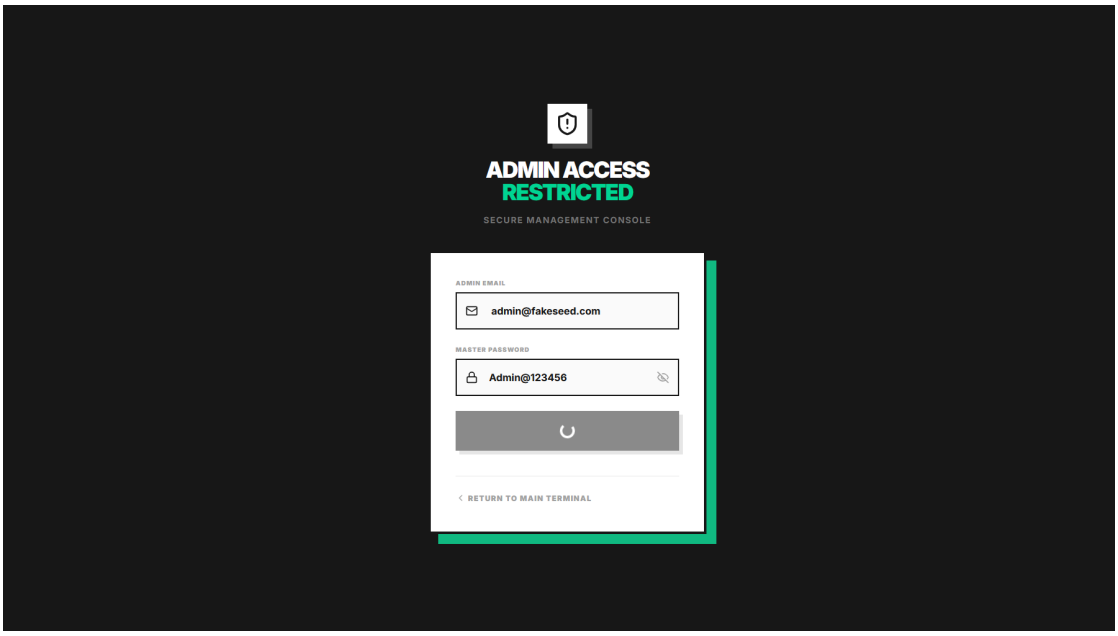

Figure 9.12: Secure admin login requiring multi-factor authentication.

9.8.2 Farmer Portal and AI Seed Quality Assessment Workflow

Once authenticated, farmers are redirected to a personalized dashboard containing historical verification records, analytical statistics, and regional agricultural alerts (Figure 9.13). Profile parameters and verification badges can be managed within the user profile interface (Figure 9.14).

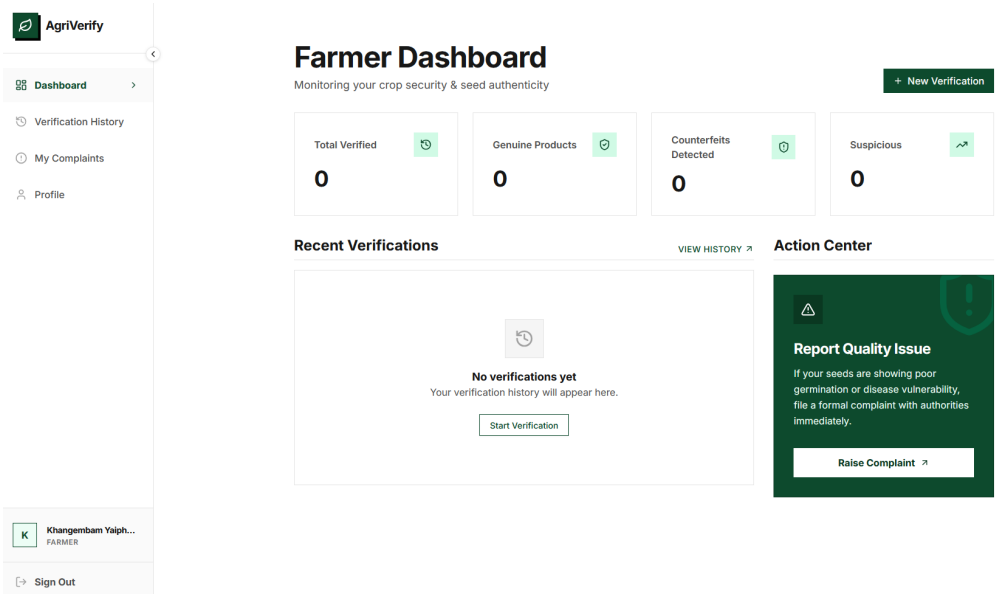

Figure 9.13: Farmer dashboard interface displaying verification statistics and history.

User details can be modified via the profile configuration interface (Figure 9.15). The

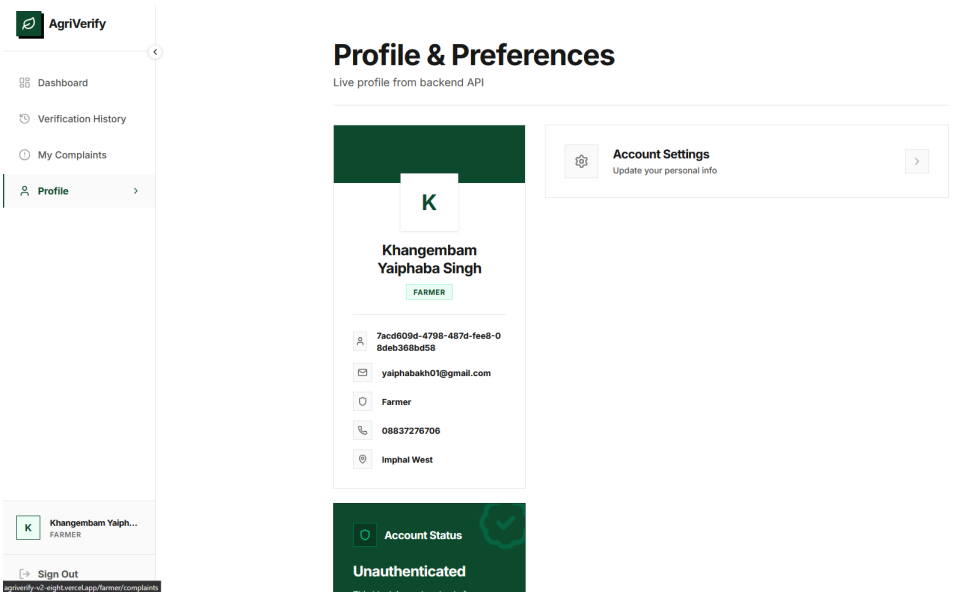

Figure 9.14: Farmer profile configuration interface with verification achievements.

seed assessment workflow is initiated by capturing and submitting high-resolution images of the front and back of the seed packaging using the mobile-optimized camera capture interface (Figure 9.16).

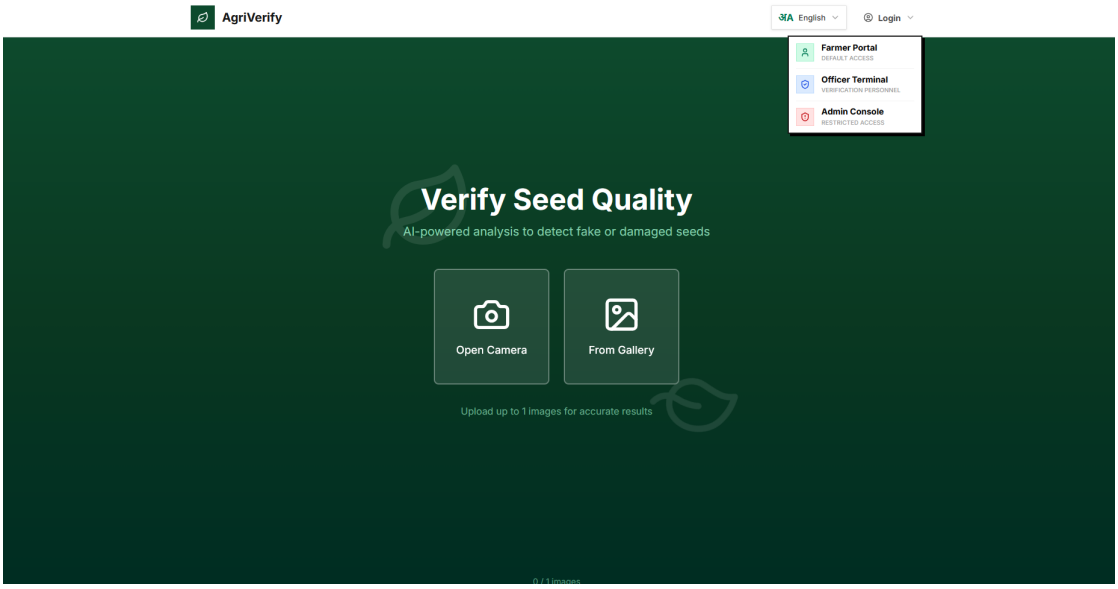

Figure 9.15: User profile modification interface.

Following image processing, the model generates a comprehensive quality report. If the seed batch is certified as pure, the interface displays extracted physical characteristics and purity metrics (Figure 9.17). Conversely, if defects or counterfeit markers are detected, a warning interface highlights quality assessment confidence and offers remedial instructions (Figure 9.18).

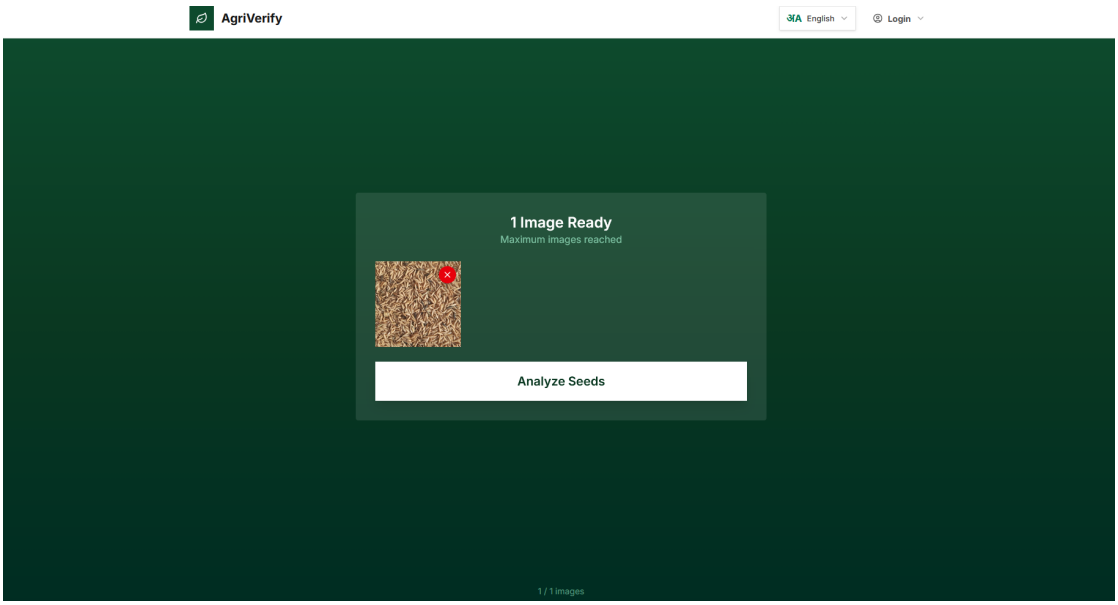

Figure 9.16: Mobile camera interface for uploading seed packaging images.

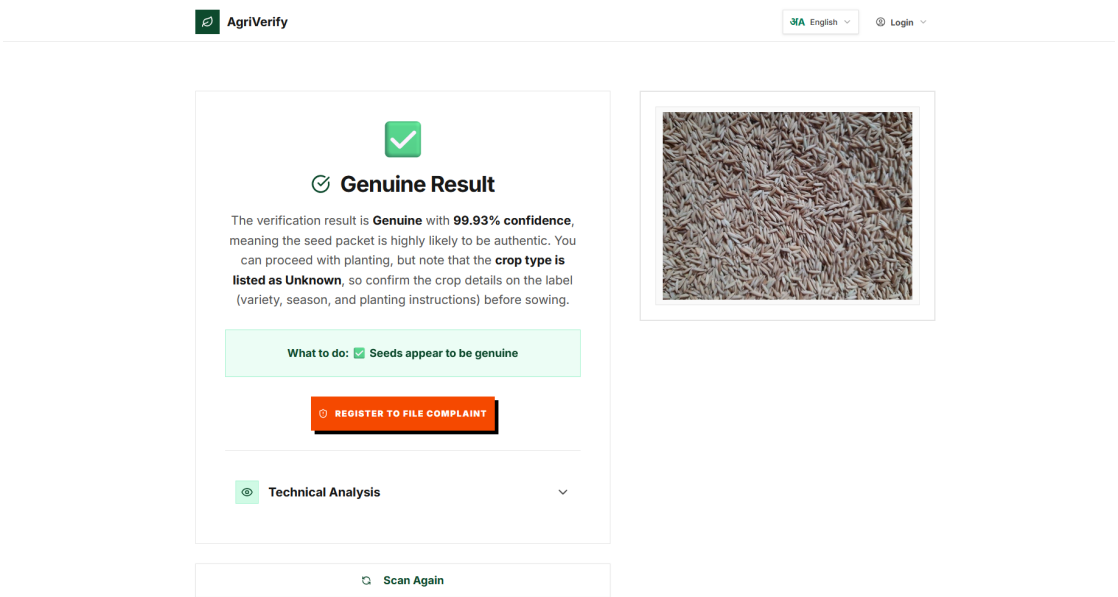

Figure 9.17: Verification result indicating pure seed batch with morphological measurements.

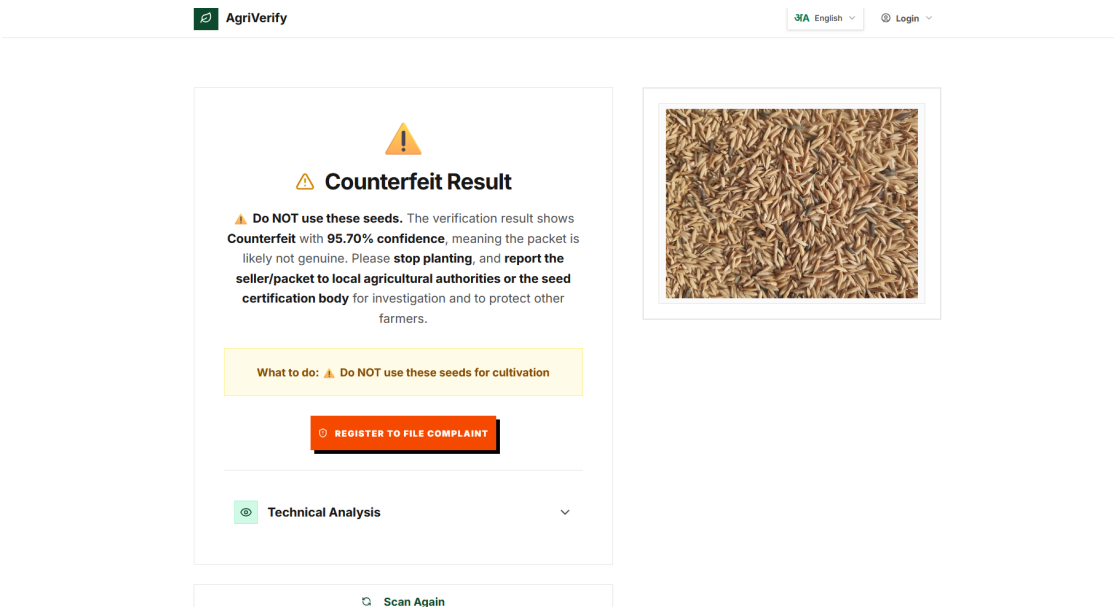

Figure 9.18: Verification alert for counterfeit or low-quality seed batch.

9.8.3 Grievance Redressal and Officer Investigation Interfaces

In instances of quality failure or suspected counterfeit distribution, farmers can lodge formal complaints detailing the batch source and severity (Figure 9.19). The status and resolution progress of these submissions can be monitored via the complaint queue interface (Figure 9.20).

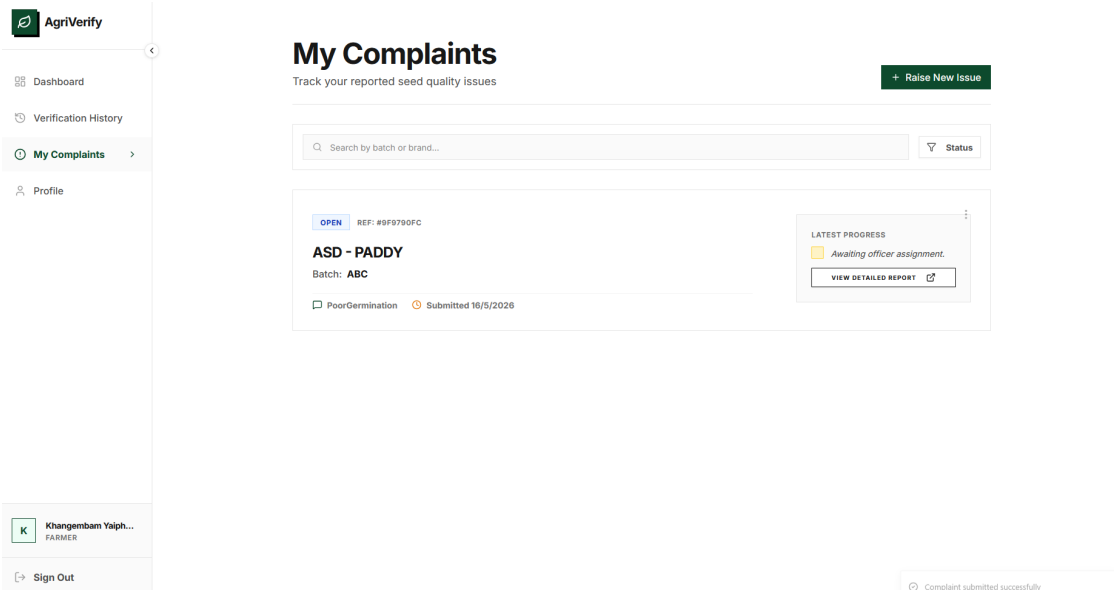

Figure 9.19: Grievance registration interface for lodging seed quality complaints.

Agricultural officers access a dedicated monitoring dashboard summarizing regional complaints and workload distribution metrics (Figure 9.21). Officers can query,

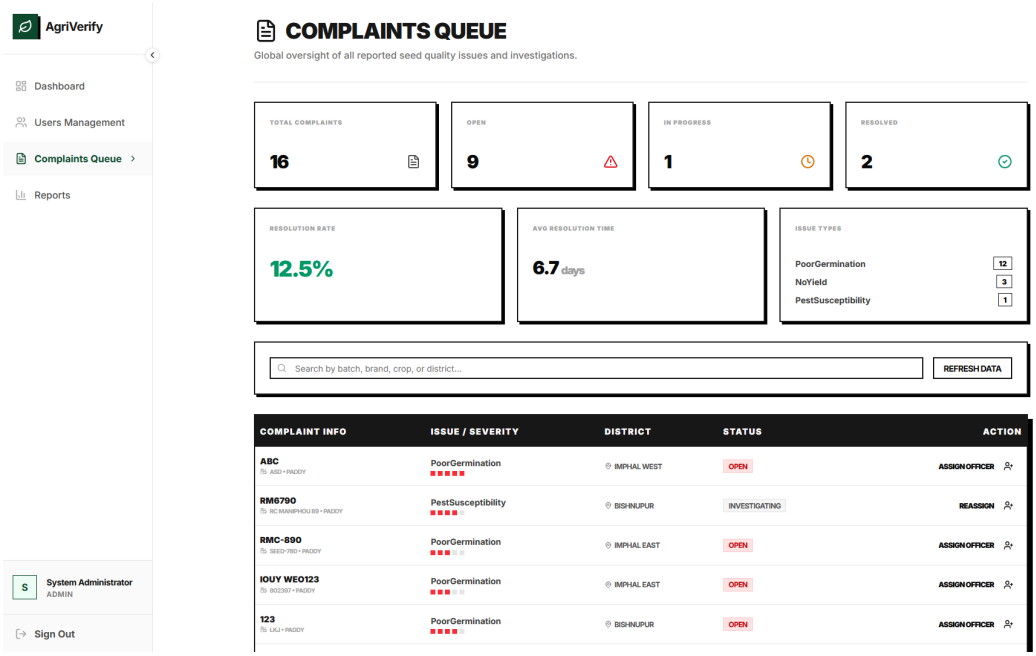

Figure 9.20: Farmer tracking interface for monitoring grievance status.

filter, and prioritize the investigation queue based on severity, date, and location (Figure 9.22). Selecting a specific complaint displays the detailed investigation pane, allowing officers to examine user-submitted image evidence and update the resolution status (Figure 9.23).

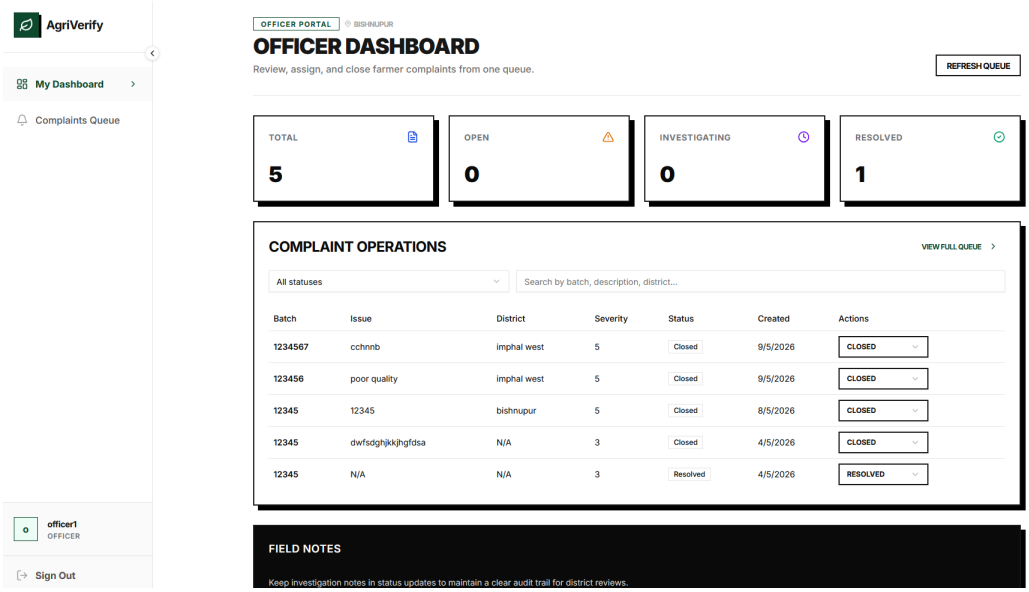

Figure 9.21: Agricultural officer dashboard presenting regional complaint metrics.

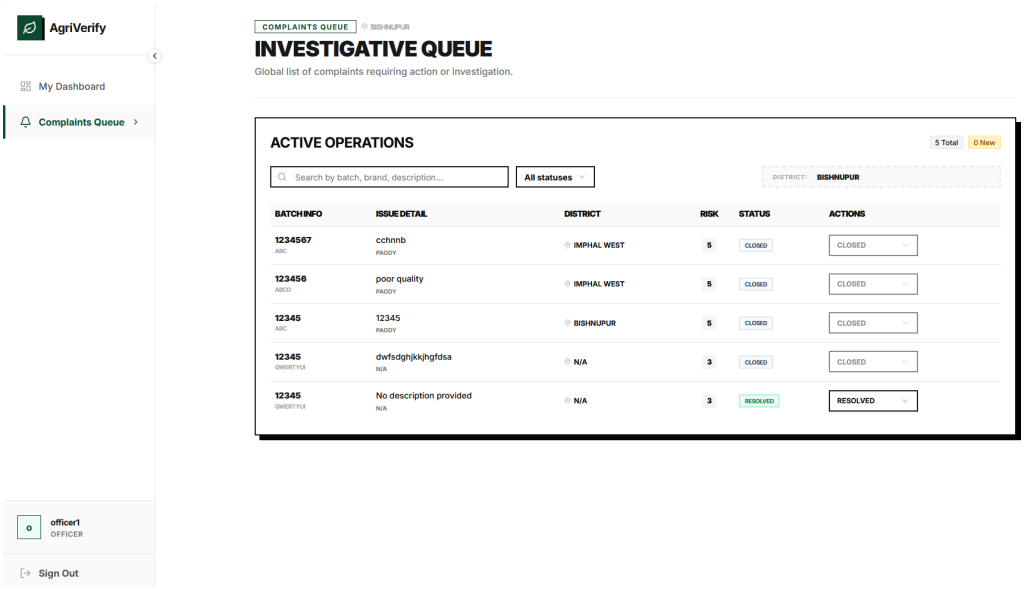

Figure 9.22: Filtered and prioritized grievance investigation queue for officers.

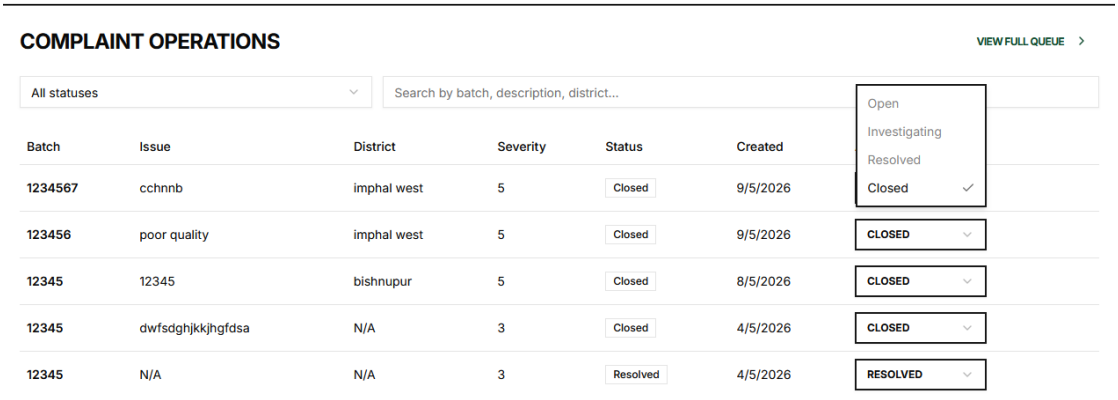

Figure 9.23: Officer grievance detail pane featuring user evidence and resolution controls.

9.8.4 Platform Administration, User Governance and Telemetry

Platform administrators monitor system telemetry, API response times, active sessions and database load via the real-time administrator dashboard (Figure 9.24). Administrative user governance, including account creation and role modification, is managed through the user governance portal (Figure 9.25).

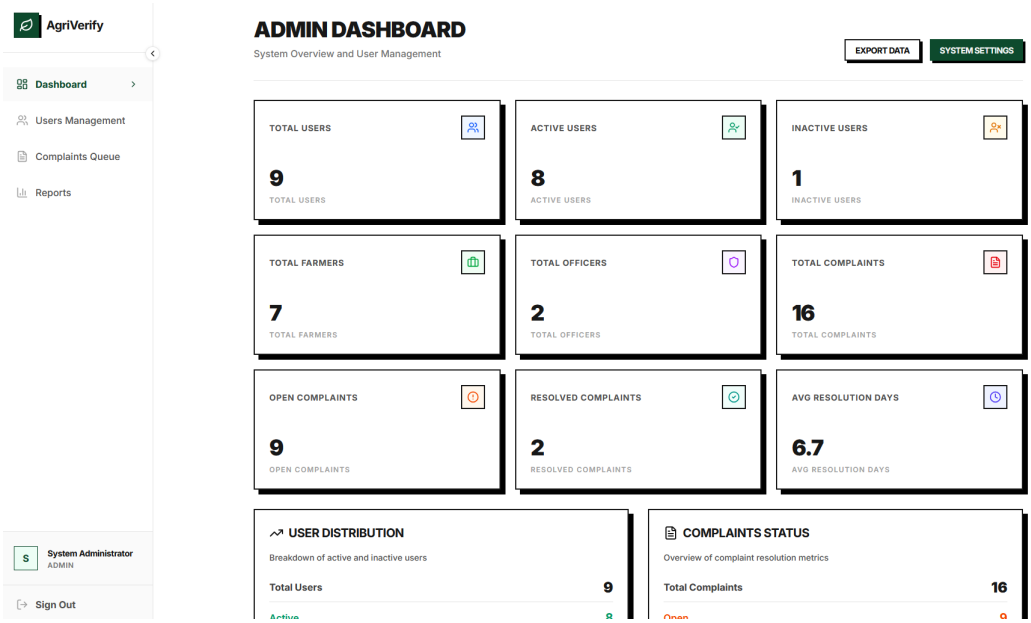

Figure 9.24: Administrator real-time system performance telemetry dashboard.

System analytics are visualized through reporting dashboards that highlight verification trends and low quality seed detection rates over time (Figure 9.26). Geographical visualization dashboards map the distribution of counterfeit seed incidents across different administrative districts, aiding policymakers in identifying counterfeit hotspots (Figure 9.27).

9.9 Challenges Encountered and Mitigation Strategies

The development and deployment of AgriVerify encountered several technical challenges, which were addressed using specific mitigation strategies:

- **Data Scarcity:** The training dataset comprised only 1,563 images, introducing a high risk of model overfitting. This constraint was mitigated by utilizing transfer learning with pre-trained ImageNet weights, applying data augmentation to simulate sample diversity, and employing weight decay to regularize model complexity [36], [39].

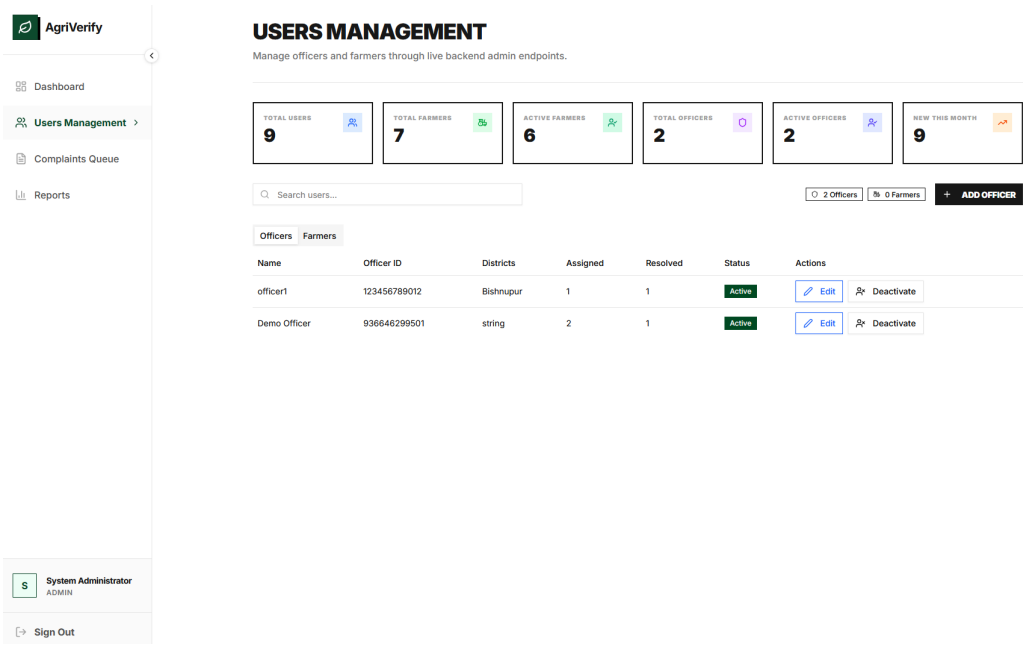

Figure 9.25: User administration and access control interface.

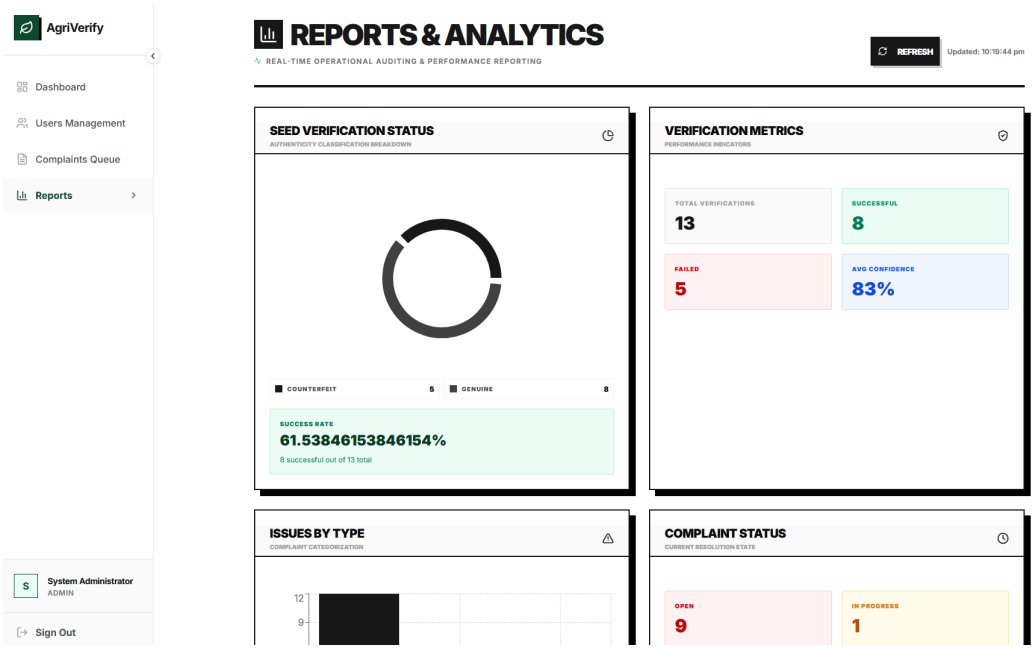

Figure 9.26: Analytical reporting dashboard showing quality trends and system statistics.

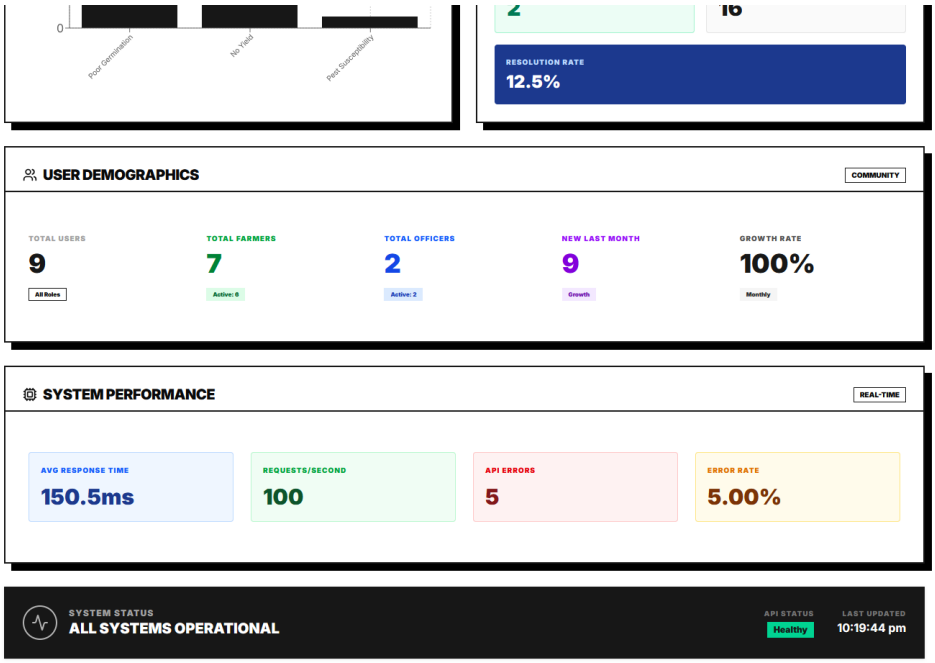

Figure 9.27: District-wise geospatial hot-spot mapping of counterfeit occurrences.

- **Fine-Grained Feature Similarity:** Discriminating between mechanically damaged seeds and high-quality counterfeit packaging required high sensitivity. The system mitigated this by combining visual texture analysis with Optical Character Recognition (OCR) to extract text features from the packaging labels, providing multi-modal confirmation [35].

- **Environmental Illumination Variations:** Field photographs taken by farmers under uncontrolled ambient lighting conditions could degrade quality assessment accuracy. To resolve this, automatic preprocessing algorithms adjust image contrast and brightness before inference, and training images were augmented with random color jittering to enhance model robustness.

- **Resource Constraints and Model Scale:** Deploying a standard ResNet50 model (approximately 100 MB) on edge servers introduces latency and memory bottlenecks. This was mitigated by implementing half-precision floating-point quantization (FP16) and model optimization, reducing memory footprints without compromising accuracy [15].

Chapter 10

Conclusion And Future Work

10.1 Summary of Work

We developed AgriVerify to address a practical problem: smallholder farmers in rural regions do not have an easy way to verify paddy seed quality before planting. Traditional seed inspection methods require sending samples to specialized labs, which is expensive, slow, and inaccessible to average farmers. This system bridges that gap by combining a deep learning model for visual inspection, packaging text extraction (OCR), and an administrative complaint workflow into a single platform.

The system's architecture reflects these requirements. We built a Next.js web application that provides role-based interfaces for farmers, agricultural officers, and administrators. The backend was developed using ASP.NET Core following Clean Architecture principles, ensuring that data validation and business rules are separated from database or web concerns. We served the deep learning model using a FastAPI microservice, while Azure AI services were integrated to run OCR on packaging labels and generate plain-language feedback reports.

To evaluate seed quality, we trained a ResNet50 classifier using transfer learning on a dataset of healthy, damaged, and impure paddy grains. We used image preprocessing and augmentation techniques—including rotation, normalization, and brightness adjustments—to ensure the model remains robust under the poor lighting conditions typical of rural fields. The model measures key physical traits such as grain length, width, circularity, and color distribution.

When a user runs a scan, the backend combines the seed classification results with the batch details extracted from the packaging label. It then uses Azure OpenAI to compile these findings into a simple, non-technical summary. If a farmer discovers a seed batch has failed to germinate after sowing, they can log a complaint through

the platform. These reports are sent directly to regional agricultural officers, who can review the complaints, record their investigations, and flag problematic seed batches. This provides a transparent feedback loop to help track seed fraud in local markets.

10.2 Key Contributions and Findings

The primary contribution of this work lies in the development and deployment of a localized deep learning pipeline for paddy seed classification that bridges scientific certification parameters with field-level agricultural practice. By training a ResNet50 classifier on seed specimens collected in collaboration with local research institutions, the system successfully categorizes samples into genuine and impure batches. Unlike simple binary image classification systems, this model extracts key morphometric descriptors—including grain length, aspect ratio, and husk uniformity—and compares them against established Distinctness, Uniformity, and Stability (DUS) standards. This approach provides smallholder farmers with a scientifically grounded quality assessment directly in the field, helping to eliminate the subjectivity and inconsistency associated with manual seed inspections.

Furthermore, this project demonstrates the feasibility of orchestrating heterogeneous AI services within a unified, real-time transaction workflow. The backend architecture consolidates multiple specialized services—combining Azure Blob Storage, a FastAPI machine learning microservice, Azure AI Vision OCR, and the Azure OpenAI Service—into a single transactional request. By extracting packaging batch numbers via optical character recognition and passing the combined classification metrics to a large language model, the system synthesizes unstructured analytical statistics into a clear, plain-language advisory summary. This design shows how complex, multi-service machine learning workflows can be simplified to deliver immediate, actionable feedback to users.

Finally, the system addresses the operational challenges of deploying advanced software services in resource-constrained rural environments. To ensure usability over unstable 3G and 4G networks in districts like Senapati and Tamenglong, we implemented client-side image compression in the browser to minimize upload payloads without losing morphological detail. This mobile-first optimization, combined with a Next.js server-side API proxy, shields sensitive authorization credentials while bypassing client network bottlenecks. The deployment of this containerized architecture across Vercel and Render demonstrates a low-cost, scalable framework that empowers agricultural communities to combat seed counterfeiting and report market anomalies directly to regional enforcement authorities.

10.3 Future Enhancements

While the current system implements the core verification pipeline, several practical improvements would make it more viable for long-term field use. Our first priority is adding local language localization, specifically Meiteilon (Manipuri) and Kuki-Chin languages, and integrating text-to-speech audio feedback. Because many farmers in rural Manipur prefer oral communication or struggle with English interfaces, audio-guided reports would lower the barrier to adoption.

Additionally, network reliability remains a bottleneck in hilly districts. To address this, we plan to implement a Progressive Web Application (PWA) wrapper that supports offline caching. By converting our ResNet50 model weights to ONNX or TensorFlow Lite formats, we can execute basic visual classifications directly on the user's mobile device browser without making backend API calls. The application would queue verification reports and sync them with our central SQL database once the device returns to cellular coverage.

Finally, expanding the classification model requires broader dataset collection. Our current training data focuses on a limited set of local cultivars. Working with regional agricultural departments to collect and annotate samples of local heirloom varieties and common state-certified seeds like *RC Maniphou-12* would improve model robustness in different districts. We also plan to integrate physical security checks, such as scanning official government holograms or manufacturer QR codes, to run in parallel with the morphological visual assessment, creating a multi-layered verification check.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016. DOI: 10.3389/fpls.2016.01419.

[2] M. Dyrmann, H. Karstoft, and H. S. Midtiby, "Plant species classification using deep convolutional neural network," *Biosystems Engineering*, vol. 151, pp. 72–80, 2016. DOI: 10.1016/j.biosystemseng.2016.08.024.

[3] Y. Zhang, S.-j. Wang, G. Ji, and P. Phillips, "Fruit classification using computer vision and feedforward neural network," *Journal of Food Engineering*, vol. 243, pp. 123–135, 2020.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.

[5] P. Muñoz, M. Navas, *et al.*, "ICT and mobile applications for agricultural extension in low-resource settings: A systematic review," *Computers and Electronics in Agriculture*, vol. 154, pp. 234–245, 2018.

[6] C. Ni, W. Fang, Y. Cheng, *et al.*, "Classification of rice seed varieties using morphological parameters extracted from binary silhouettes," *Transactions of the ASABE*, vol. 52, no. 3, pp. 951–957, 2009.

[7] J. Liu, Q. Wang, and H. Li, "Application of VGG16 transfer learning to maize seed defect detection," *Computers and Electronics in Agriculture*, vol. 175, p. 105 566, 2020.

[8] P. Xu, X. Chen, and L. Zhang, "ResNet-50 for soybean quality grading and fine-tuning evaluation," *Agronomy Journal*, vol. 113, no. 4, pp. 3210–3221, 2021.

[9] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ: Prentice Hall, 2018, ISBN: 978-0134494166.

[10] Vercel Inc., "Next.js 14 app router documentation," Vercel, Tech. Rep., 2023. [Online]. Available: https://nextjs.org/docs.

[11] TanStack, *TanStack Query (React Query) v5 documentation*, https://tanstack.com/query/latest, Accessed: 2024, 2023.

[12] D. Kato, *Zustand: A small, fast, and scalable state management solution*, https://github.com/pmndrs/zustand, Accessed: 2024, 2023.

[13] Tailwind Labs, *Tailwind CSS framework documentation*, 2024. [Online]. Available: https://tailwindcss.com/docs.

[14] Microsoft Corporation, "ASP.NET Core 8 documentation," Microsoft, Tech. Rep., 2023. [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/.

[15] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8026–8037, 2019. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[16] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000. [Online]. Available: https://opencv.org.

[17] S. Ramírez, "FastAPI documentation," Tiangolo, Tech. Rep., 2023. [Online]. Available: https://fastapi.tiangolo.com.

[18] OWASP Foundation, "OWASP Top Ten 2021: The ten most critical web application security risks," Open Web Application Security Project, Tech. Rep., 2021. [Online]. Available: https://owasp.org/Top10/.

[19] S. Ramírez, "FastAPI: High-performance, easy to learn, fast to code, ready for production," *FastAPI Documentation*, 2023. [Online]. Available: https://fastapi.tiangolo.com.

[20] D. Merkel, "Docker: Lightweight linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, p. 2, 2014.

[21] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[22] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.

[23] Google Cloud, "Google Cloud Run documentation: Fully managed serverless container platform," Google, Tech. Rep., 2023. [Online]. Available: https://cloud.google.com/run/docs.

[24] M. Salim *et al.*, "Microservices-based system for automated data collection, sentiment analysis, and visualization in video game reviews across multiple platforms," in *IEEE Conference Publication*, IEEE, 2024.

[25] S. S. Singh, N. K. Mishra, and S. A. Singh, "Impact of online communication platforms on knowledge and adoption behaviour of farmers in eastern uttar pradesh: A study on major crops," *Journal of Experimental Agriculture International*, 2026.

[26] Meta Platforms, Inc., *React documentation: Concurrent rendering*, 2024. [Online]. Available: https://react.dev/.

[27] TanStack, *TanStack Query: Asynchronous state management*, 2024. [Online]. Available: https://tanstack.com/query/latest.

[28] W. G. Masue, D. Ngondya, and T. S. Kondo, "Quantifying vulnerabilities: A systematic review of the state-of-the-art web-based systems," *Journal of ICT Systems*, vol. 2, no. 1, pp. 72–86, 2024. DOI: 10.56279/jicts.v2i1.51.

[29] Vercel Inc., *Next.js documentation: App router & proxy middleware*, 2024. [Online]. Available: https://nextjs.org/docs.

[30] Axios Contributors, *Axios: Promise-based HTTP client for the browser and Node.js*, https://axios-http.com/docs/intro, Accessed: 2024, 2024.

[31] M. Jones, J. Bradley, and N. Sakimura, "JSON web token (JWT)," Internet Engineering Task Force (IETF), Tech. Rep. RFC 7519, 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519.

[32] A. Freeman, *Pro ASP.NET Core 3: Develop Cloud-Ready Web Applications Using MVC, Blazor, and Razor Pages*. Berkeley, CA: Apress, 2020. DOI: 10.1007/978-1-4842-5440-0.

[33] J. Copard, *Microsoft Azure Architecture Patterns*. Birmingham, UK: Packt Publishing, 2020.

[34] F. Pedregosa, G. Varoquaux, A. Gramfort, *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html.

[35] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016. DOI: 10.3389/fpls.2016.01419.

[36] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7.

[37] R. Müller, S. Kornblith, and G. E. Hinton, "When does label smoothing help?" *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019. [Online]. Available: https://proceedings.neurips.cc/paper/2019/hash/f1748d6b0fd9d439f71450117eba2725-Abstract.html.

[38] J. Walke, "React: A JavaScript library for building user interfaces," *Meta Open Source*, 2013. [Online]. Available: https://reactjs.org.

[39] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, pp. 1097–1105, 2012. [Online]. Available: https://proceedings.neurips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html.